

ST. MARY'S UNIVERSITY
FACULTY OF INFORMATICS
DEPARTMENT OF COMPUTER SCIENCE



**CARGOS AND LOGISTICS TRANSPORT BIDDING
MANAGEMENT SYSTEM**

BY

Name	ID
1. Hermella Gebrehawaryat	RAD/0388/2013
2. Hikma Umer	RCD/1215/2013
3. Julia Kalegze	RCD/0880/2013
4. Abdulhafiz Shemsu	RCD/2004/2013

ADDIS ABABA

**CARGOS AND LOGISTIC TRANSPORT BIDDING
MANAGEMENT SYSTEM**

BY:

**Hermella Gebrehawaryat, Hikma Umer, Julia Kalebze, and Abdulhafiz
Shemsu**

**A SENIOR PROJECT DOCUMENT, SUBMITTED TO
THE DEPARTMENT OF COMPUTER SCIENCE, FACULTY
OF INFORMATICS, IN PARTIAL FULFILLMENT OF THE
REQUIREMENTS FOR THE DEGREE OF BACHELOR OF
SCIENCE IN COMPUTER SCIENCE**

JUNE 2025

ADDIS ABABA, ETHIOPIA



ST. MARY'S UNIVERSITY

**CARGOS AND LOGISTIC TRANSPORT BIDDING
MANAGEMENT SYSTEM**

FACULTY OF INFORMATICS

DEPARTMENT OF COMPUTER SCIENCE

APPROVED BY THE COMMITTEE OF EXAMINERS

Dr. Tibebu Tulu

Department Head

Signature

Biniyam Abebe

Adviser

Signature

Internal Examiner

Signature

External Examiner

Signature

CERTIFICATE OF AUTHENTICATED WORK

This is to certify that this project “CARGOS AND LOGISTICS TRANSPORT BIDDING MANAGEMENT SYSTEM” was carried out by group of students of computer science at St. Mary’s University as a requirement for senior project by:

Group Members:

Name	Signature
Abdulhafiz Shemsu	_____
Hermella Gebrehawaryat	_____
Hikma Umer	_____
Julia Kalegze	_____

The matter embodied in this project is authentic and is genuine work done by the student and has not been submitted whether to this university or any other university / institute for the fulfillment of the requirement of any course of study. This Project proposal was reviewed and approved (for examination) by:

Advisor Name

Signature

Date

Biniyam Abebe

ACKNOWLEDGEMENTS

We would like express our sincere thanks and appreciation to everyone who has helped us to complete this project successfully

We express our deepest gratitude to our adviser, Binyam, for his invaluable guidance, expertise, and unwavering support throughout this research . His insightful feedback and encouragement were instrumental in shaping this work. We also extend our appreciation to our colleagues and peers for their constructive discussions and contributions, which greatly enriched this study

We would like to express our gratitude to those truck owners who helped us on the project by polite answers , and everyone else among you for contributing in some manner to the completion and current form of this project and class.

Finally, we want to express our sincere gratitude to everyone who has supported us, especially our advisors and semen transport Associations. We are very appreciative of your help and advice, as well as the openness with which you responded to our questions. We appreciate all you've done for us.

List of Abbreviations

Abbreviation	Definition
PC	Personal Computer
UI/UX	User Interface/ User Experience
HTTP	Hypertext Transfer Protocol
NF	Normal Form

List of figures Figure

1: Gantt Chart.....	
Figure 2:Use Case Diagram.....	
Figure 4: Sequence Diagram.....	
Figure 5: Activity Diagram.....	
Figure 6: State chart Diagram.....	
Figure 8:Deployment Diagram.....	
Figure 9: Component Diagram.....	
Figure 10: EER Diagram	

Table of Contents

Chapter 1: Introduction

1.1. Background of the Project.....	3
1.2. Statement of the Problem and Justification	3
1.3. Objective of the Project	4
1.3.1. General Objective	4
1.3.2. Specific Objective	4
1.4. Methodology	5
1.4.1. Data collection	5
1.4.2 System Development Process Model.....	5
1.4.3. Design Pattern-----	6
1.4.4. Programming Languages.....	7
1.5.Tools-----	8
1.5.1. Hardware Tools-----	8
1.5.2. Software Tools-----	8
1.6 Scope and Limitation of the Project-----	9
1.6.1. Scope of the Project-----	9
1.6.2. Limitations of the Project-----	9
1.7. Significance of the Project-----	10
1.8. Feasibility Study-----	11
1.8.1. Technical Feasibility -----	11
1.8.2. Economic Feasibility-----	12
1.8.3. Operational Feasibility-----	12
1.9. Risk Assessment -----	13
1.9.1. Risk -----	13

1.9.2 Assumptions-----	14
1.9.3 Constraints -----	14
1.10. Work Breakdown -----	14-15
Chapter 2: Business Area Analysis and Requirement Definition	
2.1. Introduction-----	16
2.2. Current System-----	16
2.2.1. Players of the Existing System-----	17
2.3. Business Rules-----	17
2.4. Proposed System-----	17
2.4.1. Overview-----	17
2.4.2. Requirement Definition-----	18
2.4.2.1. Functional Requirements-----	18
2.4.2.2. Essential Use Case Modeling-----	19
2.4.2.3. Essential User Interface Prototyping-----	19
2.4.2.4. Collaboration Modeling-----	20
2.4.2.5. Nonfunctional Requirements-----	20
2.4.3. System Models-----	21
2.4.3.1. Use Case Model-----	21
2.4.3.1.1. Actor Description-----	23
2.4.3.1.2. Use Case Diagram-----	30
2.4.3.1.3. Use Case Description-----	33
2.4.3.2 Sequence Diagram-----	39
2.4.3.3 Activity Diagram-----	41
2.4.3.4 State chart Diagram -----	43
2.4.3.5 Class Diagram-----	44

2.4.3.6 Data Model-----	
2.4.3.6.1 ER Diagram-----	
2.4.3.7 User Interface Prototyping-----	

Chapter 3: System Design

3.1. Introduction.....	
3.2. Purpose of the system.....	
3.3. Design goals.....	
3.4. Current software architecture (if applicable).....	
3.5 Proposed Software architecture	
3.5.1. Subsystem decomposition.....	
3.5.2. Component diagram.....	
3.5.3. Deployment diagram.....	
3.5.4. Persistent data management.....	
3.5.5 Detailed Database Design.....	
3.5.5.1. Relational tables.....	
3.5.5.2. Normalization.....	
3.5.5.3. EER.....	
3.5.5.4. OO-Relational mapping.....	
3.5.6. Access control and security.....	
3.5.7. Global software control.....	
3.5.8. Boundary conditions.....	

Chapter one

1. Introduction

1.2. Background of the project

In today's fast-paced world, the need for efficiency, and communication in the logistics industry, especially in terms of how bids are handled, contracts are awarded, and how stakeholders can interact with each other. These challenges often lead to delays, disputes and dissatisfaction for all parties involved. To address these challenges, the Cargo and Logistics Transport Bidding Management System was conceived.

The main goal of the project is to create a digital platform that simplifies the bidding process for freight transport services. It allows truck owners to register, monitor existing bids and connect with winning associations. On the other hand, users are given a platform to bid for freight transport opportunities, submit bids and negotiate. The system allows for a transparent payment process where associations deposit their funds within a certain time frame and the lowest bidder wins the contract.

The platform also ensures quality of service throughout the process by introducing a rating system to assess the performance of carriers. Investors, organizations, or government entities can provide ratings based on on-time, cargo handling, and communication, which are essential to maintaining high service standards. These ratings help prospective bidders assess the reliability of their partners and hold them accountable for their services.

By incorporating these features, the Cargo and Logistics Transport Bidding Management System seeks to create a more transparent, competitive, and user-friendly environment for freight transportation. It eliminates traditional weaknesses, enhances communication, and builds trust

between all parties, ultimately contributing to the smooth operation of the logistics industry. With this system, the organization aims to transform how logistics and cargo transport services are managed, making them more efficient, secure, and responsive to today's market demands.

1.2. Statement of the Problem

The cargos and logistics transportation industry has become increasingly complex due to the rapid growth of global trade, e-commerce, and industrial activity. Despite this growth, the sector still suffers from poor regulation of freight transportation. One of the main issues is the lack of a streamlined system to connect carriers and shippers. Currently, there is no centralized platform for stakeholders to easily connect, bid, and finalize contracts, which leads to delays and confusion in the process.

Another key problem is the lack of transparency in how freight contracts are awarded. Bidding processes are often opaque, with no clear way to track how decisions are made or ensure that contracts are awarded to the most competitive and trustworthy bidders. This creates dissatisfaction among stakeholders as to whether bids are conducted fairly or whether all interested parties have an equal opportunity to compete.

The communication gap between stakeholders is also evident in the current logistics model. Carriers and shippers often find themselves lacking adequate means to communicate directly with each other or to resolve concerns in real time. This lack of continuous communication can lead to misunderstandings, missed opportunities and inefficiencies, all of which can affect the overall efficiency of the logistics process.

Ensuring service quality in the entire transportation process is also a challenge. Without a clear mechanism to monitor the performance of both carriers and shippers, it is difficult to hold any party accountable for delays, damage to goods or other issues. This lack of accountability erodes trust between stakeholders and reduces the reliability of the service provided.

1.3. The objective of the project

1.3.1 General Objective

The general objective of this project is to develop a digital platform that streamlines the bidding process for cargo and logistics transport services, enhancing transparency, efficiency, and stakeholder engagement.

1.3.2. Specific Objective

- **User Registration System and Onboarding System:** Design and implement a secure and user-friendly registration system that enables to create and manage accounts, submit

necessary documentation (e.g., business licenses, permits...), and complete onboarding requirements to ensure platform integrity and eligibility.

- **Posting feature for shippers:** Allow shippers to create and publish cargo transport requests with detailed shipment information including origin, destination, cargo type, weight, and volume.
- **Bid evaluation and comparison features:** Ensure that the system can automatically compare submitted bids based on criteria such as price and rating and present them in a ranking format.
- **Real-Time Notifications:** Integrate real-time notifications and updates for users regarding bid status, shipment tracking, and other relevant information to improve communication.
- **Feedback Mechanism:** Establish a feedback mechanism that allows users to provide input on their platform usage experience through reviews and ratings, which will be used to refine and enhance the system's usability.
- **Data Analytics Utilization:** Utilize data analytics capabilities to gather insights on user behavior, bidding patterns, and market trends. This approach will facilitate informed decision-making and drive continuous improvement in the platform's effectiveness, benefiting all stakeholders involved.

1.4. Methodology

1.4.1. Data Collection

Data gathering methodology pertains to the particular techniques and strategies employed to collect and analyze data for a research project. When examining cargo and logistics transport bidding management systems, various methods can be utilized for data collection; in this study, we opted for questionnaires and interviews. Below is an explanation of how these methods could be applied in a bidding system research study:

Questionnaires: A questionnaire is a structured survey that consists of a set of predetermined questions directed at a group of respondents.

Interviews: Interviews involve one-on-one discussions between a researcher and a respondent, where the researcher poses a series of open-ended questions to collect detailed information and insights.

1.4.2 System Development Process Model

System development process models are structured approaches that guide the planning, creation, testing, and deployment of software systems. These models provide a framework for managing

the complexities of software development, ensuring that projects are completed efficiently, on time, and within budget. By following a specific model, development teams can better manage requirements, design, implementation, and maintenance, while also facilitating communication among stakeholders.

For the cargoes and logistics transport bidding management system, we have selected the Agile development model. The Agile model is particularly well-suited for this project due to its iterative nature, which allows for continuous feedback and adaptation throughout the development process. Given the specific objectives outlined, such as user interface design, bid comparison features, and real-time notifications, the Agile model enables the development team to prioritize these features based on user feedback and changing market demands. This flexibility is crucial in the transport industry, where user needs and technological advancements can evolve rapidly.

The Agile model aligns well with the project's goals of enhancing transparency, efficiency, and stakeholder engagement. By breaking the project into smaller, manageable iterations, the team can focus on delivering functional components of the system incrementally. This approach not only allows for regular testing and validation of features like the user-friendly interface and data analytics integration but also fosters collaboration among stakeholders, ensuring that the final product meets their expectations and requirements. Overall, the Agile model supports the project's aim of creating a robust and user-centric digital platform for cargoes and logistics transport services.

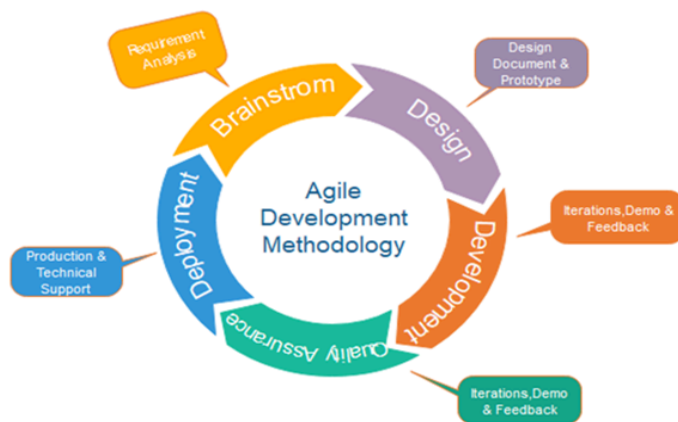


Figure 1 Illustration of Agile SDLC

1.4.3. Design Pattern

In developing the digital platform, we have chosen to follow the Model-View-Template (MVT) design pattern, which is a variant of the Model-View-Controller (MVC) architecture specifically tailored for the Django framework. The MVT pattern divides the application into three interconnected components: the Model, which handles the data and business logic; the View, which manages the user interface and presentation logic; and the Template, which is responsible for rendering the HTML content that users interact with.

The decision to adopt the MVT design pattern stems from its inherent advantages in promoting a clean separation of concerns, which is essential for maintaining a scalable and maintainable codebase. In this architecture, the Model handles all data-related operations, ensuring that the business logic is encapsulated and easily testable. The View acts as an intermediary, managing the flow of data between the Model and the Template, while the Template focuses solely on presenting the data to the user in an engaging manner. This separation allows for easier updates and modifications to individual components without disrupting the overall functionality of the application.

In general Model-View-Template (MVT) pattern operates by allowing the Model to define the data structure and business rules, while the View processes user requests and interacts with the Model to retrieve or manipulate data. The Template then takes this data and formats it for display to the user. This flow ensures that the user interface remains dynamic and responsive to user interactions, which is crucial for features such as bid comparison, real-time notifications, and data analytics integration. By promoting a clear separation of concerns, changes in one component do not adversely affect the others, fostering a modular approach to development. Additionally, Django's built-in features, such as its Object-Relational Mapping (ORM) for database interactions and its templating engine for rendering HTML, streamline the development process. This allows us to focus on building a robust and user-friendly platform that meets the specific objectives outlined for the cargo bidding management system, including user-friendly interface design and effective feedback mechanisms.

Overall, the MVT design pattern not only aligns with the technical requirements of our project but also enhances collaboration among team members by providing a clear structure for development. This approach will ultimately contribute to the successful realization of our goal to create a robust and efficient cargo bidding management system that meets the needs of its users while fostering transparency and engagement in the transport industry.

1.4.4. Programming Languages

For the development of the cargoes and logistics transport bidding management system, we have chosen the following programming languages:

Frontend	
HTML:	Used for structuring the content of the web application.
CSS:	Employed for styling and layout, ensuring an attractive and user-friendly interface.
JavaScript:	Utilized for adding interactivity and enhancing user experience on the client side.
Backend	
Python (Django Framework):	Selected for server-side development.

Table of Programming Languages

We chose Python as the backend programming language primarily due to its simplicity and readability, which facilitate rapid development and easier maintenance of the codebase. Python's extensive libraries and frameworks, particularly Django, provide robust tools for building web applications efficiently.

Django's built-in features, such as its Object-Relational Mapping (ORM) for database interactions, authentication mechanisms, and templating engine, streamline the development process, allowing us to focus on implementing the specific objectives of our project. The framework follows the Model-View-Template (MVT) design pattern, which promotes a clear separation of concerns, making it easier to manage and scale the application as it grows.

Additionally, Django's strong community support and comprehensive documentation enhance its appeal, providing resources and solutions for common challenges encountered during development. The framework's emphasis on security, scalability, and rapid development aligns well with our goals of creating a robust, efficient, and user-friendly cargo and logistics transport bidding management system. Overall, Python and Django together offer a powerful combination that meets the needs of our users while improving operational efficiency in the transport industry.

1.5. Tools

1.5.1. Hardware Tools

PCs:	Development will primarily be done on desktop and laptop computers with sufficient
------	--

	processing power and memory to run development environments and testing tools.
Mobile Devices:	Testing the web application on various mobile devices to ensure responsiveness and usability across different screen sizes.
Servers:	A dedicated server or cloud-based server (e.g., AWS) for hosting the application and database during development and production.
Networking Equipment:	Routers and switches to facilitate local network connectivity for development and testing environments.

Table of Hardware Tools

1.5.2. Software Tools

Framework	Django Framework: A high-level Python web framework that encourages rapid development and clean, pragmatic design.
Database	PostgreSQL: A powerful, open-source object-relational database system used for storing application data.
Development Tools	Visual Studio Code: A popular IDE for writing and debugging code, with support for various programming languages and extensions.
Version Control	GitHub: A platform for hosting Git repositories, facilitating collaboration and version control.

Testing Tools	Postman: A tool for testing APIs, allowing developers to send requests and analyze responses.
Diagramming Tools	Draw.io: An online diagramming tool for creating various types of diagrams, including class diagrams and UI mockups.
UI/UX Design Tools	Figma: A collaborative interface design tool for creating UI mockups and prototypes.

Table of Software Tools

1.6. Scope and Limitation of the Project

1.6.1. Scope of the Project

Where does it work?

- The system will operate in the cargo and logistics transportation industry, facilitating the bidding process for cargo transport services.

For whom?

- The primary users will include shippers (those needing cargo transport) and carriers (transport service providers).

When?

- The system will be available for use 24/7, allowing users to place bids, track shipments, and receive notifications at any time.

On what platform?

- The application will be a web-based platform, accessible via desktop and mobile browsers.

Included expected MAIN features:

User-friendly interface for easy navigation and interaction.

Bid comparison features to allow shippers to compare bids from multiple carriers.

Real-time notifications for users regarding bid status and shipment tracking.

Comprehensive database for storing bidding data, user profiles, and transaction history.

Feedback mechanism for shippers to provide input on their experience.

1.6.2. Limitations of the Project

What features that are normally expected aren't covered by your project?

- The system may not include advanced features such as AI-driven predictive analytics or machine learning capabilities for bid optimization.

Where does it not work?

- The system may not be suitable for regions with limited internet connectivity or for users who do not have access to modern web browsers.

When doesn't it work?

- The system may experience downtime during maintenance or updates, which could temporarily limit access for users.

Who cannot use it?

- Users without internet access or those who are not familiar with using web applications may find it challenging to use the system.

On what platforms can a user not use it?

- The application will not be available as a standalone desktop application or mobile app; it will only be accessible through web browsers.

1.7. Significance of the Project

Business Value:

- **Increased Efficiency:** streamlines the bidding process for cargo and logistics services, reducing time and manual effort.
- **Cost Reduction:** By providing transparent pricing and competitive bidding, businesses can find the most cost-effective transport solutions.
- **Operational Improvement:** Facilitates better resource allocation and management for transport companies, enhancing overall operational efficiency.

Social Impact:

- **Transparency:** Enhances the transparency of the bidding process, fostering trust between shippers, carriers, and other stakeholders.
- **Fair Competition:** Promotes equal opportunities for all carriers, improving the competitive landscape within the transport industry.

Research Contribution:

- **Innovation in Logistics:** Advances digitalization in the cargos and logistics transport sector by providing a data-driven platform for the bidding process.
- **Data Analytics for Industry Insights:** Offers valuable insights into bidding trends, user behavior, and market dynamics, contributing to the understanding of the transport ecosystem.

Shippers (Businesses/Customers):

- **Bid Comparison:** Easily compare multiple bids from carriers based on pricing, service levels, ensuring the best choice for their needs.
- **Feedback Mechanism:** Rate and review carriers, helping future customers make informed choices and ensuring a high standard of service.

Carriers (Transport Providers):

- **Increased Visibility:** Gain exposure to a wider pool of potential clients by participating in a competitive and transparent bidding process.
- **Improved Communication:** Stay informed with real-time notifications on bids, updates, and feedback, helping them respond promptly and efficiently.

1.8. Feasibility Study

The feasibility study for this project will assess the practicality and potential success of developing a digital platform for cargo and logistics transport services. The types of feasibility studies considered include technical feasibility, to evaluate the platform's technology requirements, economic feasibility, to analyze cost and financial viability, and operational feasibility, to ensure the platform can be effectively implemented and maintained within the transport industry.

1.8.1. Technical Feasibility

The development of a digital platform to streamline the bidding process for cargo and logistics transport services is technically feasible based on the required hardware, software resources, and the capabilities of the development team.

The project requires robust hardware, including cloud-based servers for user traffic and ensure high availability. For software, front-end such as HTML, CSS, JAVASCRIPT will create a user-friendly interface, while Python (Django) will manage server-side logic and database interactions. A relational database like PostgreSQL will securely store user data and bids.

The chosen tools and technologies are appropriate for the project's objectives. Modern web frameworks will ensure a seamless user experience, while cloud infrastructure allows for scalability.

The success of the project depends on the student's willingness to learn and what they have learned so far.

1.8.2. Economic Feasibility

Economic feasibility refers to a project's capacity to be financially viable and cost-effective. The following factors enhance the economic viability of this project:

Development Cost

- Software: Free/open-source tools (Django, PostgreSQL) reduce expenses.
- Hardware: Existing computers and servers can host the system; no specialized hardware required.

Cost Saving

- The online system offers substantial cost savings by removing the need for manual paperwork and reducing expenses related to printing, document storage.

Resource Saving

- It saves users both time and money by allowing them to submit documents digitally.

1.8.3. Operational Feasibility

- **Current Inefficiencies:** The existing system is manual, fragmented, and inefficient, causing delays and miscommunication. The lack of digital tools hinders communication, reduces efficiency.

- Proposed Operational Improvements: Bidding is intended to address these operational issues through digitization and automation. The proposed system aims to:

- Automate and streamline processes.
- Provide a centralized platform for all stakeholders.
- Enhance communication and coordination.

1.9. Risk Assessment

1.9.1. Risk

In conducting a comprehensive risk assessment for the development, it is crucial to identify potential risks that could impede the successful completion of the project. Below are the key risks associated with this objective along with proposed mitigation strategies:

Data Security and Privacy:

Risk: The digital platform may be vulnerable to data breaches or privacy violations, jeopardizing the confidentiality and integrity of bidding information.

Mitigation: Implement robust data security measures such as encryption, access controls, and regular security audits to safeguard sensitive data. Comply with data protection regulations and industry standards to mitigate the risk of data security incidents.

User Adoption Risks:

Risk: Users may be resistant to adopting the new platform due to familiarity with existing processes or skepticism about its benefits.

Mitigation: Develop a comprehensive user onboarding process that includes training sessions, tutorials, and support resources. Highlight the platform's advantages through targeted marketing campaigns and user testimonials to build trust and encourage adoption.

System Downtime:

Regular Maintenance: Schedule routine maintenance and updates for both hardware and software to minimize the risk of failures.

Backup Systems: Establish backup systems and protocols to ensure continuity in case of hardware or software failures, including data backups and alternative access methods.

Regulatory Changes:

Compliance Monitoring: Stay informed about potential changes in laws and regulations by establishing a compliance monitoring team or subscribing to relevant legal updates.

Flexible System Design: Build the system with flexibility in mind, allowing for quick adjustments to accommodate new regulations without significant overhauls.

1.9.2 Assumptions

- Stakeholder Engagement: We assume that stakeholders will actively participate and offer valuable insights and feedback throughout the development of the system,
- Team Collaboration: We assume that all team members will work collaboratively, communicate effectively, and manage their time efficiently to meet project deadlines and achieve our shared objectives.
- Compliance with Standards: We assume that the Bidding System will adhere to all necessary requirements and standards established by our department and relevant regulatory bodies.
- The university's lab resources (e.g., servers) will be available for testing.

1.9.3. Constraints

- o Time Constraint: Project must be completed before the academic year ends (fixed deadline).
- o Scope Constraint: The project must remain within the defined scope and objectives, ensuring that all features and functionalities align with the initial project specifications.
- o Technology Constraint: The System must utilize the designated technologies and frameworks as specified in the project requirements, limiting the options for development tools and platforms.

1.10. Work Break Down

Task Name	Start Date	End Date	Estimated Duration	Description
Project Definition	Mar 11, 2025	Mar 22, 2025	~2 weeks (12 days)	Initial planning phase: define goals, scope, key deliverables, and stakeholders.
Requirement Gathering	Mar 21, 2025	Nov 1, 2025	~7.5 months (226 days)	Identify functional/non-functional requirements through meetings and documentation.
System Design	Jul 1, 2025	Jul 31, 2025	~1 month (31 days)	Create system architecture, UI/UX mockups, and technical specifications.
Development Phase	Aug 1, 2025	Sep 30, 2025	~2 months (61 days)	Implement frontend UI, develop backend logic, and ensure security best practices are followed.
First Release Cycle	Oct 1, 2025	Oct 31, 2025	~1 month (31 days)	Develop core modules; conduct unit testing and internal review.
Second Release Cycle	Nov 1, 2025	Nov 30, 2025	~1 month (30 days)	Final development sprint; polish features and enhance stability.
Testing & Debugging	Dec 1, 2025	Dec 30, 2025	~1 months (30 days)	Perform various testing types , load testing, and document bugs and fixes..
Submission	Jan 1, 2026	Jan 1, 2026	1 day	Official hand-off or deployment of the final version.

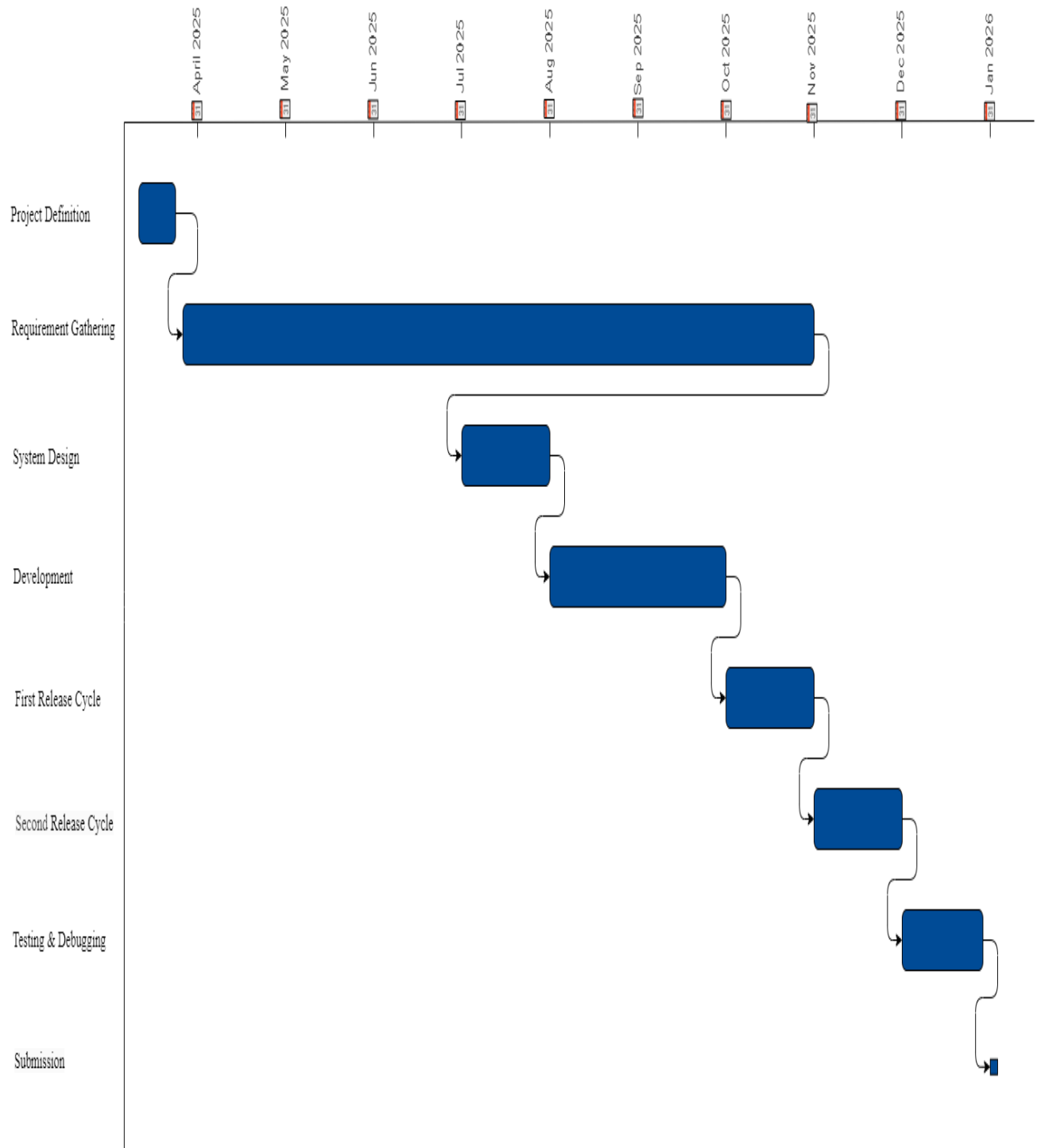


Fig of Gantt Chart

2. Business Area Analysis and Requirement Definition

2.1. Introduction

The system aims to streamline the process of connecting shippers who have cargo transport needs with carriers capable of fulfilling those requests through a competitive bidding model. By digitizing and automating key interactions—such as posting cargo bids, submitting offers, verifying identities, and processing payments—the platform seeks to eliminate inefficiencies, reduce operational delays, and increase trust among stakeholders.

The analysis focuses on identifying business processes, user roles, current operational challenges, and the specific functional and non-functional requirements necessary to develop a scalable and secure system. This foundation guides the overall system design and ensures alignment with business goals and user expectations.

2.2. Current System

Currently, the logistics and cargo transport sector operates largely through manual processes or disparate legacy systems, such as email communications, phone calls, and spreadsheets for coordinating shipments and negotiating transport prices. There is no centralized digital platform that efficiently connects shippers and carriers, leading to various operational inefficiencies and lack of transparency.

Key challenges in the existing system include:

- **Manual Matching:** Shippers must individually contact carriers or rely on intermediaries, which is time-consuming and often unreliable.
- **Lack of Competitive Pricing:** Without a transparent bidding mechanism, carriers are often selected without comparing multiple offers.
- **Verification Gaps:** Identity and document verification of users (e.g., licenses, insurance) are often informal or delayed.
- **Poor Communication Flow:** Notifications and updates regarding shipments, bid statuses, or payment approvals are handled manually, leading to missed information or delays.
- **No Performance Feedback:** There is little to no structured mechanism for users to rate or review one another after transactions.

2.2.1. Players of the Existing System

The current cargo and logistics transport system includes these main players:

Carriers: individuals or groups who offer transportation services for goods.

- **Truck Owners:** Independent individuals who own and operate their trucks. They usually depend on intermediaries to access transport opportunities, because they do not have a license to participate in bidding.
- **Transporters:** Licensed established by one or more than people. They may own trucks or hire independent truck owners when they win a bid.
- **Transport Associations:** Licensed groups formed by multiple people. They collaborate to bid on larger contracts and resources to fulfill transport needs.

Brokers: Informal middlemen who link shippers with carriers (transporters or associations). They often lack transparency and add extra costs.

Shippers: Businesses or individuals needing cargo transport. They face challenges in finding reliable, cost-effective, and transparent services.

2.3. Business Rules

User Registration and verification: All users (shippers, carriers) register and verify their accounts using valid documentation.

Licensing Compliance: Only legally licensed shippers and carriers are authorized to Bid, participate in bidding and execute transport contracts.

Bid Posting(shippers): Only verified shippers can create bid requests with details.

Bid Submission(carrier) and Deadlines: Bids must be submitted by verified carriers within the specified timeframe.

Contract Awarding: Contracts are usually awarded based on the lowest price.

Operational Flexibility: Transporters and associations may use their own trucks or subcontract independent truck owners to fulfill awarded contracts, ensuring adaptability in resource allocation.

2.4. Proposed System

2.4.1. Overview

The proposed is a web-based digital platform that centralizes and streamlines the entire bidding process. It is designed to address inefficiencies in the current system by enabling transparency, real-time communication, and performance tracking.

Key Features of the Proposed System:

User Registration and verification: All users (shippers, carriers) register and verify their accounts using valid documentation.

Centralized Bidding Platform: Enables shippers to post cargo needs and carriers (transporters, association) to submit bids.

Real-Time Notifications: Keeps users updated statuses and system alerts.

Bid Comparison Tool: Allows the system to compare bids based on price, or user ratings.

Feedback and Rating System: Rating and feedback for carriers performance, punctuality and communication.

Database-Driven Architecture: Stores and manages data related to users, bids, transactions, and feedback efficiently.

2.4.2. Requirement Definition

The Requirement Definition is a crucial document in system development that outlines what the system should do and how it should perform. It serves as a bridge between stakeholder needs and system design, ensuring everyone involved understands the system's purpose, functionality, and boundaries.

2.4.2.1. Functional Requirements

Functional requirements define what the system should do. They describe specific behaviors, functions, or tasks that the system must perform to fulfill user needs. These requirements focus on the services the system provides, how users interact with it, and how it responds to specific inputs.

1. User Account Management

FR1: User Sign-Up

Definition: The system must allow new users (Shippers and Carriers) to register an account by providing personal details such as name, contact number, email, and role (Shipper/Carrier).

FR2: User Login

Definition: The system must allow existing users to log in using their credentials (username and password).

FR3: Identity Verification

Definition: Upon login, the system must initiate a verification process to confirm the identity of the user. This may include sending a verification code or checking administrative approval.

2. Payment Confirmation**FR4: Submit Payment Reference**

Definition: The system must allow users (Shippers and Carriers) to input a payment reference number after transferring funds via external banks.

FR5: Admin Verifies Payment

Definition: The Admin must have the ability to view, validate, and either approve or reject the submitted payment references from users.

FR6: Precondition for Bid/Offer

Definition: The system must restrict the ability to post bids or submit offers until the payment is confirmed by the Admin.

➤ **Shipper Functionalities**

FR7: Post Bid

Definition: A verified and payment-confirmed Shipper must be able to post a bid specifying shipment details such as source, destination, and weight...

FR8: Update Posted Bid

Definition: The system must allow the Shipper to update a posted bid, such as modifying shipment details.

FR9: Receive Notifications

Definition: The Shipper must receive system notifications regarding bid updates, verification status, winning bids, and payment confirmations.

Compare Offers

Definition: The system must present multiple Carrier offers to the Shipper for comparison based on price, and Carrier rating.

FR10: Rate & Review Carrier

Definition: After a successful shipment, the Shipper must be able to rate the Carrier and leave a review to help improve service transparency.

➤ **Carrier Functionalities**

FR11: View Posted Bids

Definition: The system must allow Carriers to browse and view all available bids posted by Shippers.

FR12: Submit Offer

Definition: A payment-confirmed Carrier must be able to submit an offer in response to a posted bid, including price, delivery schedule, and special conditions.

FR13: Update Submitted Offer

Definition: The Carrier must be allowed to modify a previously submitted offer only if the corresponding bid has been updated by the Shipper.

FR14: Receive Notifications

Definition: The Carrier must receive timely notifications about bid updates, verification status, winning bids, and payment confirmations.

➤ **Admin Functionalities**

FR15: Approve/Reject Verification

Definition: The Admin must be able to review user-submitted documents and approve or reject identity and payment verifications.

FR16: Manage Users and Data

Definition: The Admin must have full control to add, edit, suspend, or delete user accounts and their associated data.

FR17: Monitor System

Definition: The Admin must be able to monitor platform activities, user actions, and system health metrics to maintain security and performance.

FR18: Manage Notifications

Definition: The Admin must be able to configure and manage system-generated notifications including templates and event triggers.

Conditional Requirements

CR1: Carrier Offer Update Restriction

Definition: The system must allow Carriers to update their submitted offers only if the related bid has been modified by the Shipper.

CR2: Payment Requirement for Shippers

Definition: Shippers must confirm payment before being allowed to post bids.

CR3: Payment Requirement for Carriers

Definition: Carriers must confirm payment before being allowed to submit offers.

2.4.2.2. Essential Use Case Modeling

Essential Use Case Modeling captures what the user wants to achieve (user intentions) and what the system must do in response (system responsibilities), without focusing on UI, technology, or system design.

➤ User Registration

User Intention : I want to create an account as a Shipper or Carrier.

System Responsibility : Collect necessary data and store the user account in the database.

➤ Login

User Intention I want to securely log into the system.

System Responsibility Authenticate user credentials and allow access if verified.

➤ Identity Verification

User Intention I want to verify my identity to access the platform.

System Responsibility Accept uploaded documents for admin review. Show verification status.

➤ Submit Payment Reference

User Intention : I want to confirm my payment via bank reference.

System Responsibility : Accept and store the payment reference for admin validation. Prevent bid/offer actions until approval.

➤ Post Bid (Shipper)

User Intention : I want to post a request to transport goods.

System Responsibility : Ensure payment is confirmed, then allow bid posting with shipment details.

➤ Update Posted Bid (Shipper)

User Intention : I want to change the bid details (e.g., type, destination).

System Responsibility : Allow updates to bid if within time constraints and notify affected Carriers.

➤ **Submit Offer (Carrier)**

User Intention : I want to propose a price and conditions to transport goods.

System Responsibility : Confirm payment, then allow the offer to be submitted and associate it with the relevant bid.

➤ **Update Offer (Carrier)**

User Intention : I want to change my submitted offer details.

System Responsibility : Permit changes only if the corresponding bid has been updated by the Shipper.

➤ **Compare Offers (System Action)**

User Intention : I want to choose the best Carrier.

System Responsibility : Automatically rank and present offers based on rating, price.

➤ **Rate & Review Carrier (Shipper)**

User Intention : I want to leave feedback about the Carrier after delivery.

System Responsibility : Enable rating and review form submission linked to completed delivery.

➤ **Receive Notifications**

User Intention : I want to stay informed of updates like approvals, or bid changes.

System Responsibility : Automatically generate and deliver notifications based on system events.

➤ **Admin: Verify User & Payment**

User Intention : I want to approve or reject submitted documents and payments.

System Responsibility : Display verification requests and allow actions. Notify users of results.

➤ **Admin: Monitor Activity**

User Intention : I want to track system and user activity for issues or audits.

System Responsibility : Log actions, provide dashboards, and flag suspicious behavior.

➤ Admin: Manage Users

User Intention : I want to manage platform users.

System Responsibility : Allow user account modification, suspension, or deletion.

2.4.2.3. Essential User Interface Prototyping

Essential User Interface Prototyping refers to the early-stage, low-fidelity design of a system's interface that focuses on core user tasks and interactions, rather than visual details or aesthetic elements.

Objectives of Essential UI Prototyping

- Clarify system functional behaviors from the user's perspective
- Identify task flows and user journeys
- Gather early feedback from stakeholders
- Improve communication among users, designers, and developers
- Prevent costly changes by refining ideas early

2.4.2.4. Collaboration Modeling

Collaboration Modeling describes **how different actors (users or systems) interact with each other and the system** to fulfill a specific use case. It focuses on:

- **Actors**: Shipper, Carrier, Admin, System
- **Messages exchanged**: Requests, responses, events
- **Sequencing of interactions**
- **Cooperation between multiple parties** to achieve goals

Value of Collaboration Modeling

- **Clear Understanding**: Visualizing interactions between actors and the system provides a shared understanding among the development team and stakeholders.
- **Identify Potential Issues**: Early identification of potential bottlenecks or conflicts in the flow of information.
- **Improved Design**: Inform the design of the user interface and system architecture to ensure

smooth and efficient collaboration.

- Documentation: Serve as valuable documentation for future development and maintenance of the platform.

2.4.2.5. Nonfunctional Requirements

Performance Requirements

- The system shall respond to user actions (e.g., login, post bid, submit offer) within 2 seconds under normal load.
- The platform must support concurrent users without performance degradation.

Scalability

- The system must be horizontally scalable to accommodate an increasing number of users and transactions.
- Data storage must scale dynamically with the volume of posted bids, offers, and user information.

Availability & Reliability

- The system shall maintain uptime on a monthly basis.
- The system must recover from any crash or failure within seconds without data loss.

Security

- All data (including credentials and bid/offer details) must be encrypted in transit (HTTPS) and at rest.
- The system shall enforce role-based access control (RBAC) to restrict user actions.
- Failed login attempts shall be limited to before locking the account temporarily.

Usability

- New users should be able to complete sign-up and verification within minutes.
- The interface shall be accessible via mobile and desktop devices with a responsive layout.

- The platform must follow consistent UI/UX standards for all user roles (Shipper, Carrier, Admin).

Maintainability

- The system must be modular and maintainable, allowing components (e.g., payment or notification modules) to be updated independently.
- Code documentation and change logs shall be maintained for every system update.

Portability

- The application must run on cloud-based (e.g., AWS, Azure) and on-premise deployments.
- The system should support data backup and migration across environments without loss.

Auditability & Monitoring

- The Admin shall be able to monitor system logs, user actions, and performance metrics in real time.
- All sensitive operations (e.g., payments, account changes) must be logged and traceable.

Compliance

- The system shall comply with applicable data protection regulations (e.g., GDPR, if operating in EU regions).
- The system shall require user consent for storing personal and payment-related data.

2.4.3. System Models

System Models are abstract representations of a system that help visualize, specify, design, and document the structure and behavior of a software system. They are used during system analysis and design to understand and communicate how the system works or should work.

2.4.3.1. Use Case Model

A Use Case Model is a visual and textual representation that captures the functional requirements of a system from the perspective of users (actors). It describes:

- Who uses the system (actors)
- What they can do (use cases)
- How the system should respond to different actions

Actors (System Users)

Actor	Role Description
Shipper	Posts bids for cargo transportation
Carrier	Submits offers to fulfill bids
Admin	Manages user verification, payments, and monitors system
System	Handles background processes like notifications and comparison

Use Cases (Functionalities)

Use Case	Actor(s)	Brief Description
UC1: Register Account	Shipper, Carrier	Allows a new user to create an account with role selection
UC2: Login	Shipper, Carrier, Admin	Enables existing users to securely access the system
UC3: Verify Identity	Shipper, Carrier, Admin	Users upload identity docs; Admin verifies them

UC4: Submit Payment Reference	Shipper, Carrier	Users send bank payment reference numbers
UC5: Confirm Payment	Admin	Admin validates payment references
UC6: Post Bid	Shipper	Posts cargo bid after payment confirmation
UC7: Update Bid	Shipper	Modifies bid details before bidding deadline
UC8: View Bids	Carrier	Carrier can browse available bids
UC9: Submit Offer	Carrier	Carrier offers a price and timeline for a bid
UC10: Update Offer	Carrier	Carrier can edit an offer if Shipper updates bid
UC11: Compare Offers	System	Automatically compares offers based on criteria
UC12: Rate & Review Carrier	Shipper	Posts feedback after successful delivery
UC13: Send Notifications	System	Alerts users on events like bid updates, verifications
UC14: Manage Notifications	Admin	Controls the notification settings and content

UC15: Manage Users	Admin	Edits, disables, or deletes user accounts
UC16: Monitor Activity	Admin	Views login records, suspicious actions, and system performance logs

Use Case Relationships

Relationship	Use Case(s)	Condition
Include	UC6 → UC4	Posting a bid includes submitting payment reference
Include	UC9 → UC4	Submitting an offer requires payment submission
Extend	UC10 → UC9	Carrier can only update offer if Shipper updated the bid
Automatic Trigger	UC11, UC14 ← System	System runs these without user interaction

Tables of Use Case Model

2.4.3.1.1. Actor Description

An Actor in a system represents any user, system, or external entity that interacts with the system to perform tasks or receive services.

Actors can be:

- Primary (active) — initiates interaction with the system.
- Secondary (passive) — responds to requests or receives output.

Main Actors

Shipper

- **Type:** Primary User
- **Role:** A business or individual who wants to transport goods by posting bids.
- **Key Interactions:**
 - Registers and logs in
 - Submits payment reference
 - Posts and updates bids
 - Reviews and rates carriers
 - Receives system notifications

Carrier

- **Type:** Primary User
- **Role:** A logistics company or driver that submits offers on shipment bids posted by shippers.
- **Key Interactions:**
 - Registers and logs in
 - Submits payment reference
 - Views available bids
 - Submits and updates offers

- Receives notifications

Admin

- **Type:** System Operator
- **Role:** Manages the entire platform including users, payments, verifications, and system monitoring.
- **Key Interactions:**
 - Verifies user identity and payment references
 - Approves or rejects payment submissions
 - Manages user accounts (edit, suspend, delete)
 - Monitors system activity
 - Manages system notifications
 - Ensures secure access and compliance

System (Comparison Engine / Notification Engine)

- **Type:** Internal System Actor
- **Role:** Automatically processes tasks based on rules or triggers.
- **Key Behaviors:**
 - Compares carrier offers (price, rating)
 - Sends notifications to users (bids, approvals, etc.)
 - Enforces access control rules

2.4.3.1.2. Use Case Diagram

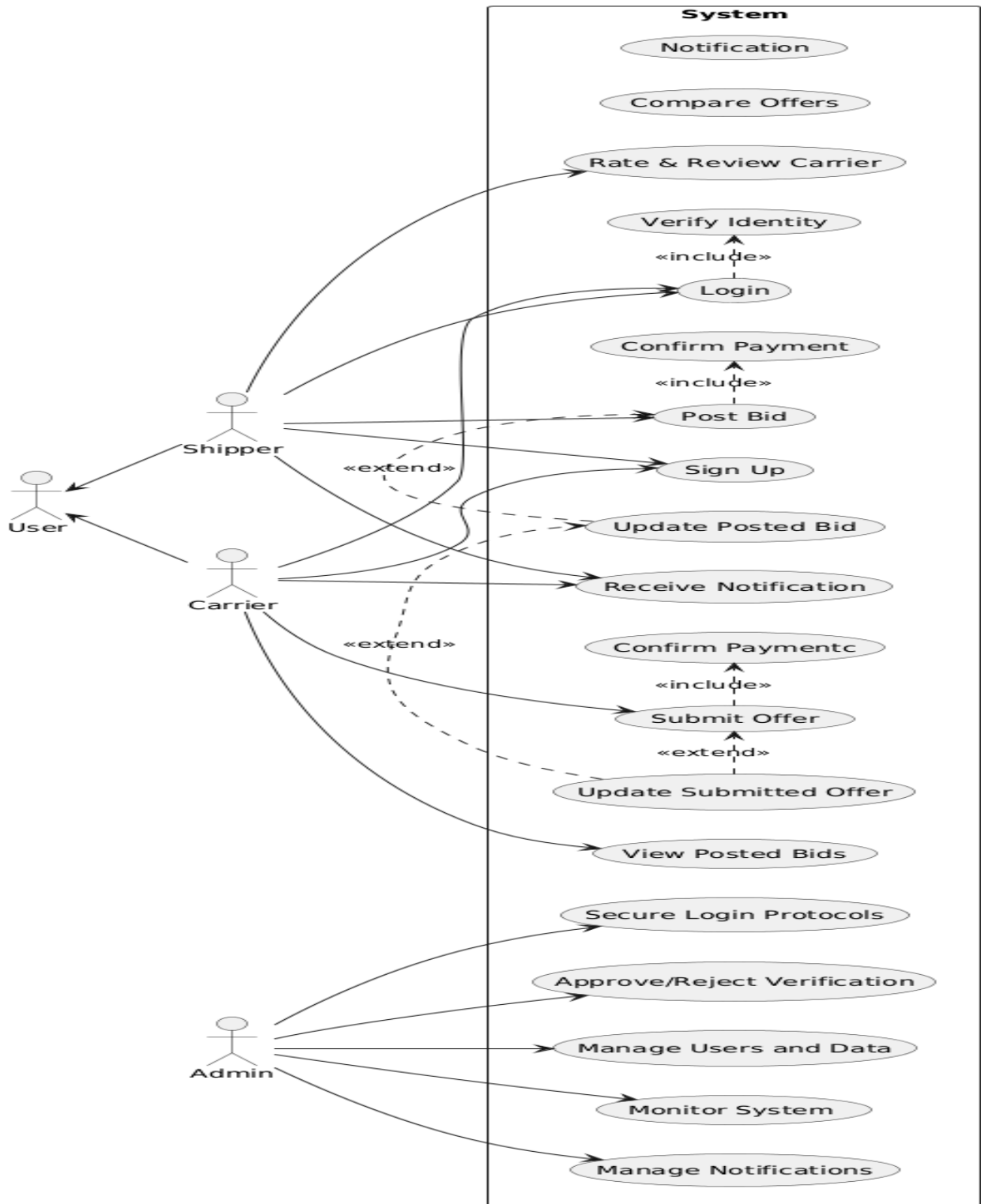


Fig of Use Case Diagram 2.1

2.4.3.1.3. Use Case Description

Use case name	Sign up
Description	A new user registers an account by providing required personal and role-specific information.
Actor	User (Shipper or Carrier)
Pre-condition	• User is not currently registered in the system. • User has access to the registration page.
Post-condition	• User account is created and stored in the system database. • Awaiting identity verification by Admin.
Basic flow	1, User opens the application and navigates to the Sign-Up page. 2, Selects user roles (Shipper or Carrier). 3, Enter personal information (name, email, phone number, password, etc.). 4, Upload identity verification documents if required. 5, Click the "Register" button. 6, System validates inputs and stores the information. 7, System marks the account as pending verification. 8, System notifies Admin for review.

Use case name	Verification Users
Description	This use case allows only eligible and authenticated users access to the platform.
Actors	Admin
Pre-condition	1, The user must have completed the sign-up process. 2, The user has submitted all required verification documents (e.g., business license, identity proof) 3, Admin receives this information for review.
Post condition	1, if approved, the user gains access to full system features. 2, If rejected, the user receives a reason for rejection and can reapply with the correct document.
Main Flow	1, Admin logs into the system dashboard. 2, Navigate to the user verification section. 3, Reviews user-submitted documents (e.g., business license, identity proof). 4, Performs background validation if needed (e.g., checks company reputation or blacklist status). 5, Approves or rejects the user based on predefined eligibility criteria. 6, System updates user status and sends notification to user.

Use case name	Login
Description	This use case allows authenticating users and allows access to their respective dashboards.
Actors	All actors (Admin, shipper, Carriers)
Pre-condition	1, The user must have already created an account(valid username/email and password). 2, The user must be verified
Post condition	Actors gain access to system functionalities based on their role.
Main Flow	1, Actor navigates to the login page 2, Actors enter their username /email and password. 3, System verifies the credentials against the stored data. 4, If credentials are correct: ▪ For Admin: Admin dashboard is displayed. ▪ For Shipper: Shipper dashboard is displayed. ▪ For Carrier: Carrier dashboard is displayed. 5, If credentials are incorrect: ▪ System displays an error message and prompts to retry or reset the password.

Use case name	User payment
Description	This use case allows shippers to pay to post bidding requests on the platform and carriers pay to participate in bids.
Actors	Carrier and Shipper

Pre-condition	The user must have signed up and verified the account. Fill information, and have valid payment information.
Post condition	Users have successfully submitted their payment information.
Main Flow	1, The user navigates to the payment section 2, The system requests the customer to enter the necessary payment. 3, The users enter the required payment information, and click the pay button.

Use case name	Post Bid
Description	This use case allows a verified shipper to post a cargo transport bid.
Actors	Shipper
Pre-condition	Shippers must be verified and confirm payment.
Post condition	Bids listed on the system for carrier participation.
Main Flow	1, Shipper logs in. 2, Navigates to "Post a Bid" section 3, Enter cargo details, destination, delivery timeline, and budget. 4, Confirms and submits the bid. 5, System publishes the bid for carrier visibility.

Use case	Submit Offer
Description	Allows carriers to express interest or submit offers for cargo transport bids.
Actors	Carriers
Precondition	Carrier must be verified and confirm payment
Postcondition	The Carrier's offer is submitted and tracked in the system.
Main flow	1, Carrier selects a bid

	2, Submits an offer including price and availability. 3, System logs the offer for shipper review. 4, Await acceptance or feedback.
--	---

Use case	Rate and review carriers
Actor	shipper
Description	Enables the shipper to rate and review the service quality of a carrier after a job is completed.
precondition	A completed transaction exists between shipper and carrier.
Postcondition	Review is added to the carrier's profile.
Main Flow	1, Shipper logs in and accesses completed 2, Selects a transaction and provides rating and feedback. 3, System stores and displays the review.

Use case	Update Bid/Offer
Actors	Carriers, Shippers
Description	This use case allows shippers to update the details of their posted bids (e.g., cargo details, destination budget), and carriers to modify their submitted offers (e.g., price) – provided the changes are made before the bid deadline.
Actors	Shipper, Carriers
Precondition	1, The actors must be posted or offer. 2, The bid must still be open, and the deadline has not passed. 3, The actors must have previously created/submitted a bid or offer.
Postcondition	1, the updated bid/offer information is saved and visible to relevant. 2, System logs the update time and maintains bid consistency.
Main Flow (Shipper)	1, carriers and shippers log in their dashboard. 2, navigate to the “My Bids” or “My offers” section. 3, Select an active bid and clicks “Update

	4, modifies bid details. 5, System checks update is within the deadline. 6, If yes, changes are saved and made visible to carriers. 7, Carriers following the bid are notified of the update.
Main Flow (Carriers)	1, carrier logs in their dashboard. 2, Navigate to “My offers” . 3, Click “Update offer”. 4, Edits offer (e.g., new price) its work, if the shipper made updates to the postedbid. 5, System checks if the bid is still open (before the deadline). 6, If allowed, the offer is updated and submitted again.

Use case	View Bids
Description	Allows a verified carrier to browse and view available cargo bids posted by shippers.
Actor	Carrier
Precondition	Carriers must be logged in.
Postcondition	List of relevant and open bids is displayed
Main Flow	1, Carrier logs into their dashboard. 2, Navigate to the “Available Bids” section. 3, Views bid details such as cargo type, destination, budget, and timeline.

Use case name	System Monitoring
Description	To observe and oversee system activities, performance, and user operations in real time.
Actors	Admin
Pre-condition	1, Admin must be authenticated and logged into the system.
Post condition	1, Admin has a clear view of system status and can take actions if anomalies are detected.
Main Flow	1, Admin logs into the system dashboard. 2, Navigate to the Monitoring Panel. 3, Views system statistics such as:

	▪ Active users ▪ Bid activity (posted bids, accepted bids) ▪ Payment transactions ▪ System uptime and error logs 4, Filters data by user role(shipper/carrier), time frame, or bid status. 5, Detects anomalies (e.g., unusually high bids, repeated failed transactions). 6, Take appropriate action (alerts a team, temporarily disables a feature, logs an incident).
--	--

Use case name	Manage Content and Data
Description	This use case allows to maintain and update platform content, and ensure data accuracy, integrity, and compliance.
Actors	Admin
Pre-condition	Admin must be authenticated and have proper access privileges.
Post condition	Content is updated, obsolete or harmful data is removed, and the system remains compliant with standards.
Main Flow	1, Admin logs into the system. 2, Navigates to the Content and Data Management Panel. 3, Performs the following actions as needed: ▪ Add/Edit/Delete informational content (FAQs, terms & conditions, policies, help guides) ▪ Update category listings (e.g., service types, transport modes).

	▪ Manage database records (user data, bid history, transaction logs). ▪ Clean old or redundant data to improve performance. 4, Verifies changes before publishing or saving. 5, Confirms updates and system logs the activity for audit purposes.
--	---

Use case name	Manage Notification
Description	To configure, send, and manage system wide notifications and alerts for users.
Actors	Admin
Pre-condition	Admin is logged into the system with proper permissions
Post condition	Notifications are delivered to targeted users or groups within the system.
Main Flow	1, Admin logs into the system dashboard. 2, Navigate to the Notification Management section. 3, Selects the type of notification: System-wide alert (e.g., maintenance notice) Targeted notification (e.g., new feature, update) Automatic alerts (e.g., bid deadlines, payment confirmation) 4, Enter the notification content (title, body, users). 5, Selects the recipients (shippers, carriers, all users). 6, schedules or sends the notification immediately. 7, System delivers the message via: ▪ In the system alert panel

Tables of use case Description

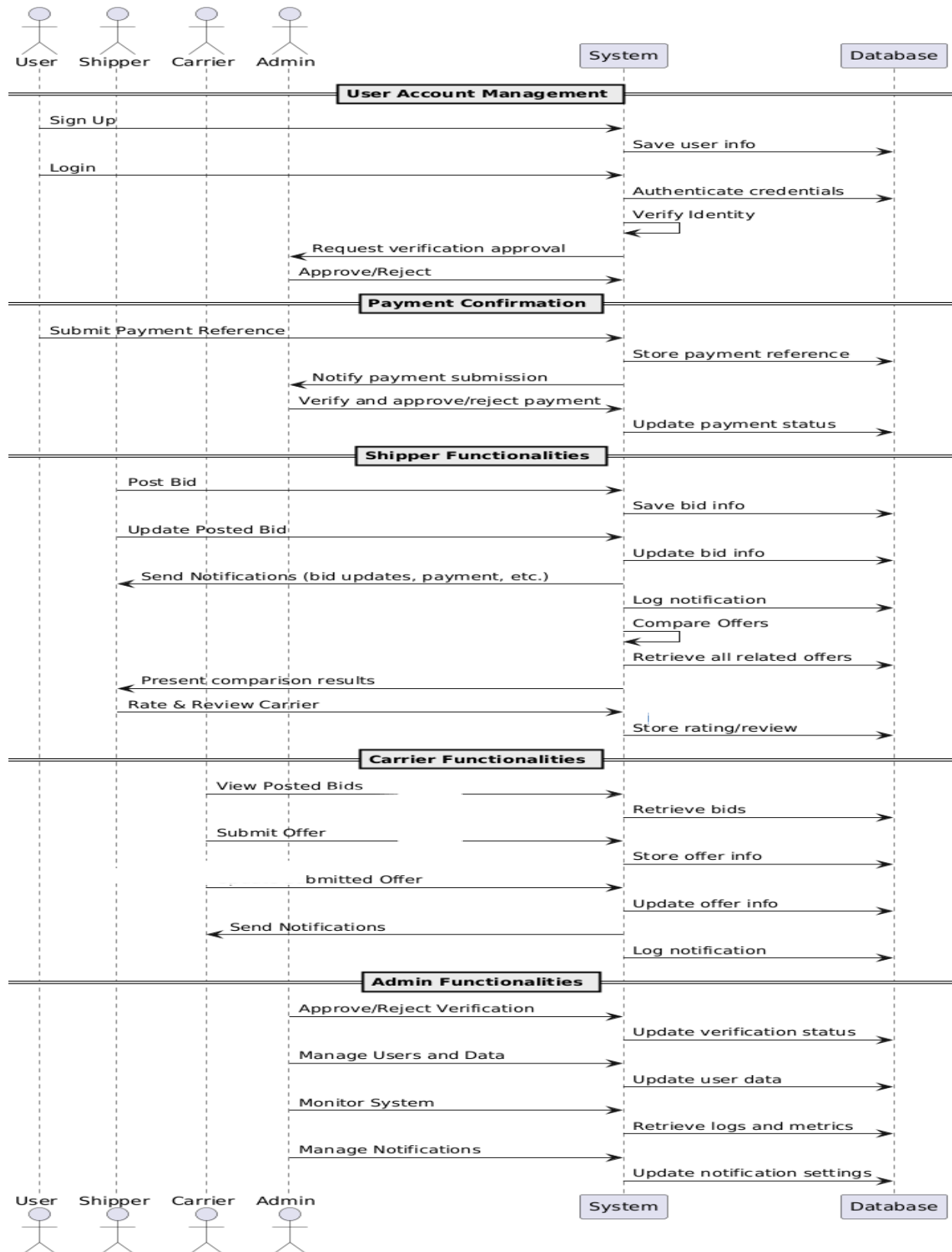


Fig of Sequence Diagram 2.3

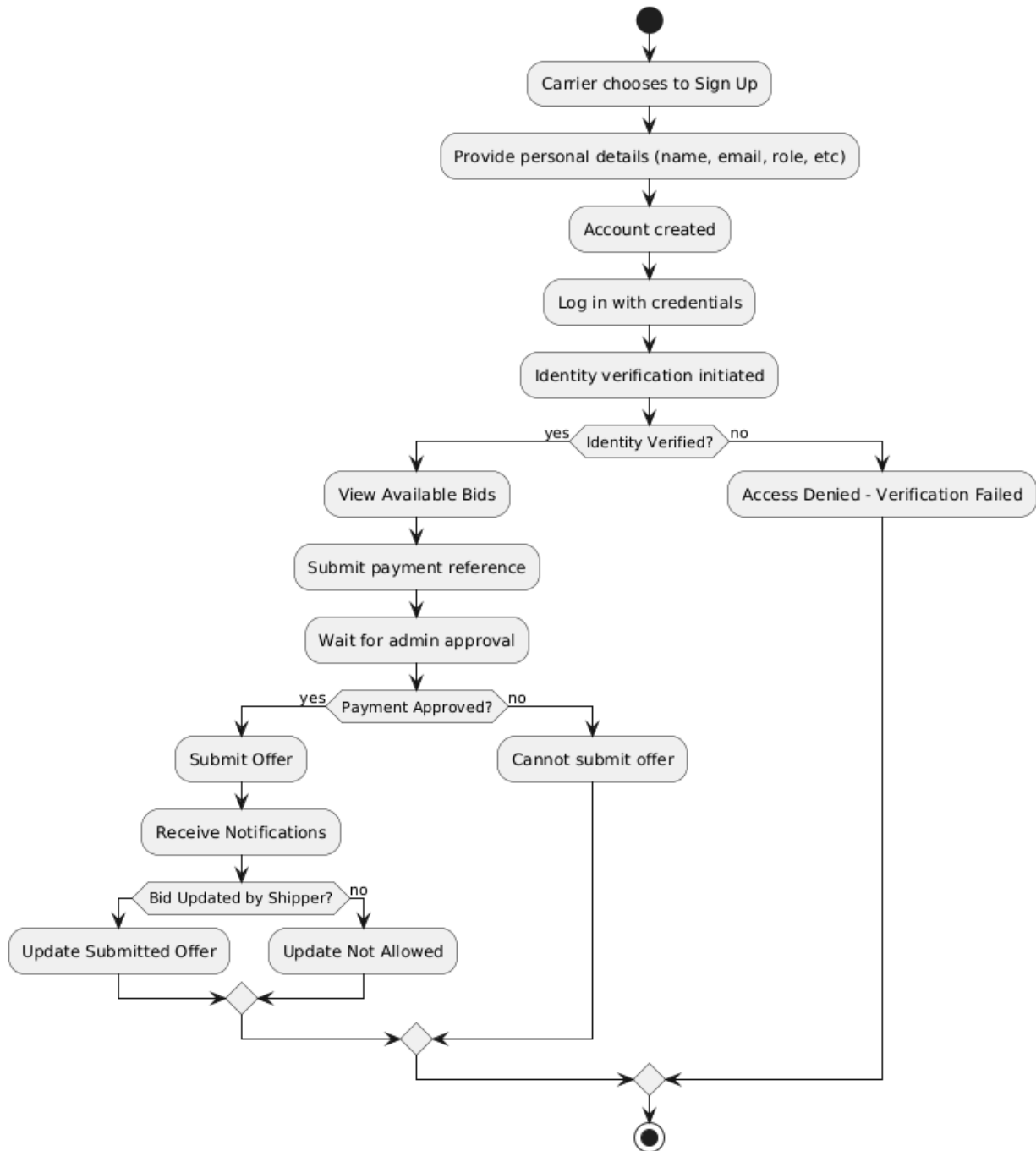


Fig of Activity Diagram for Carrier 2.3

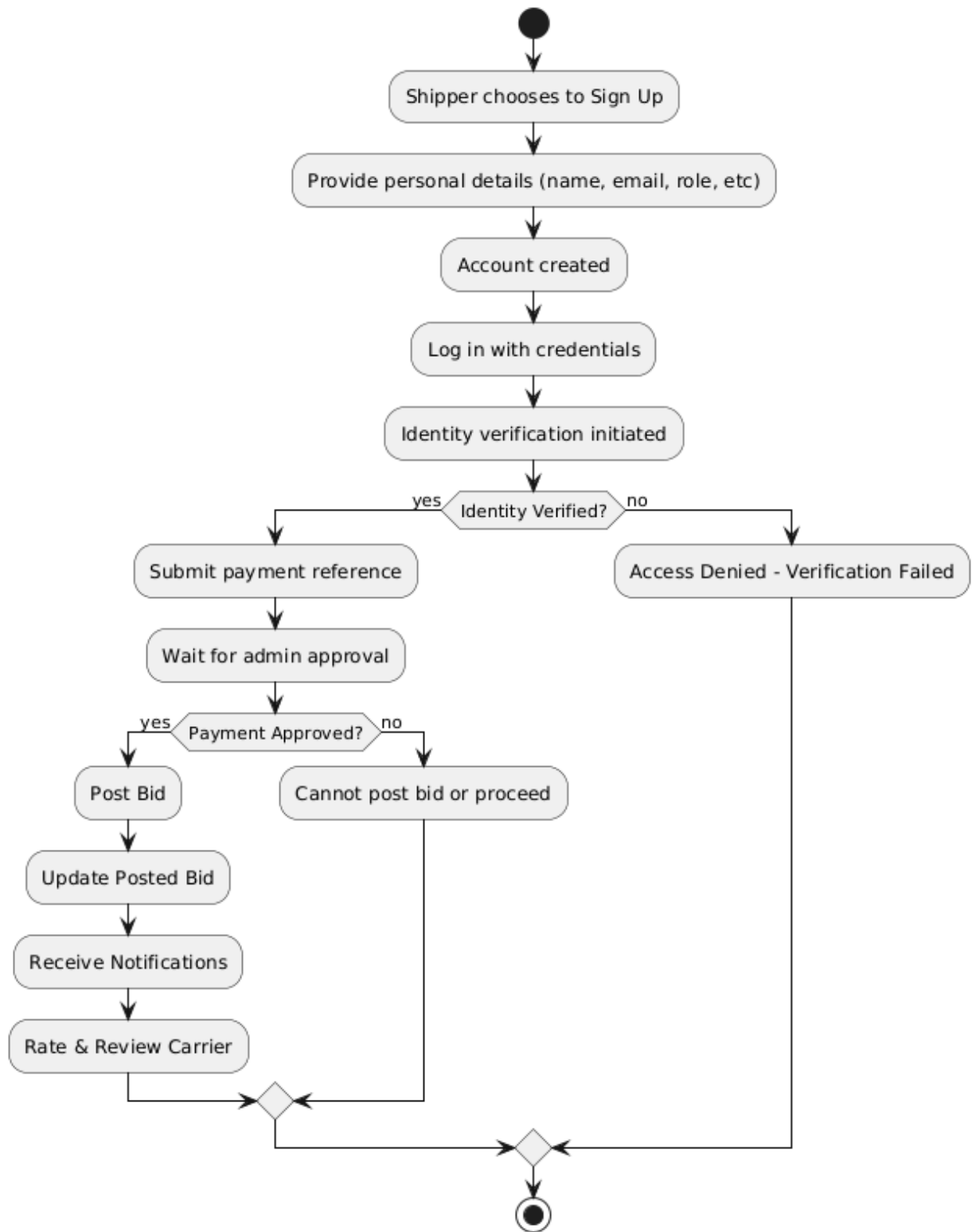


Fig of Activity Diagram for Shipper 2.3.1

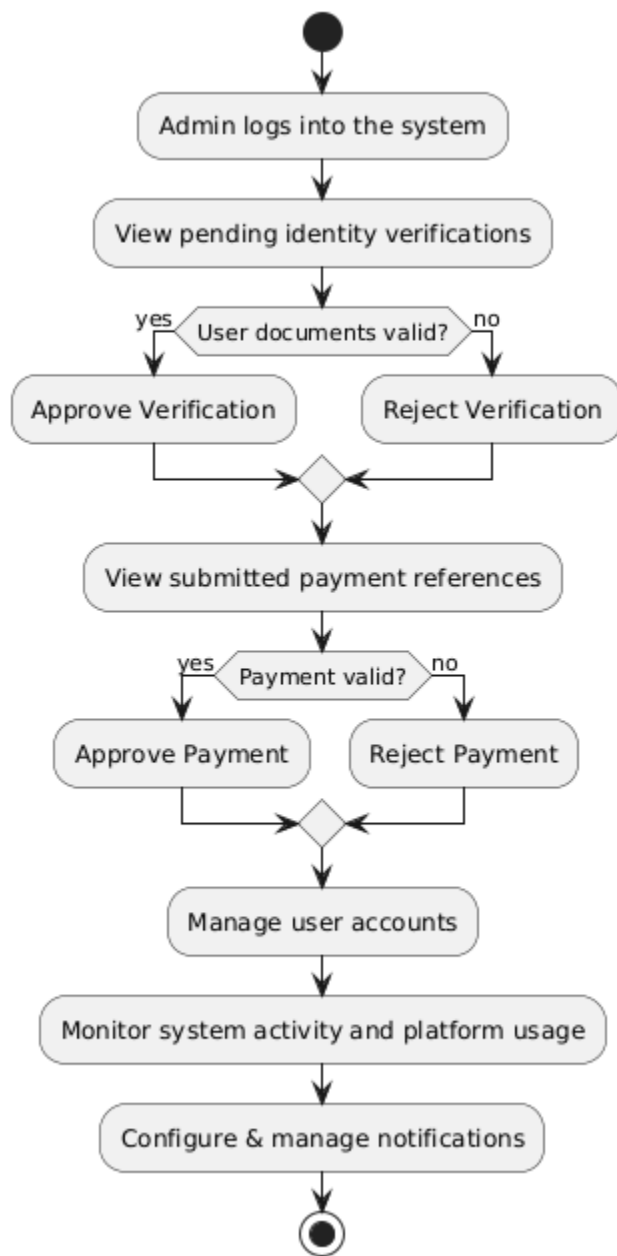


Fig of Active Diagram for shipper 2.3.2

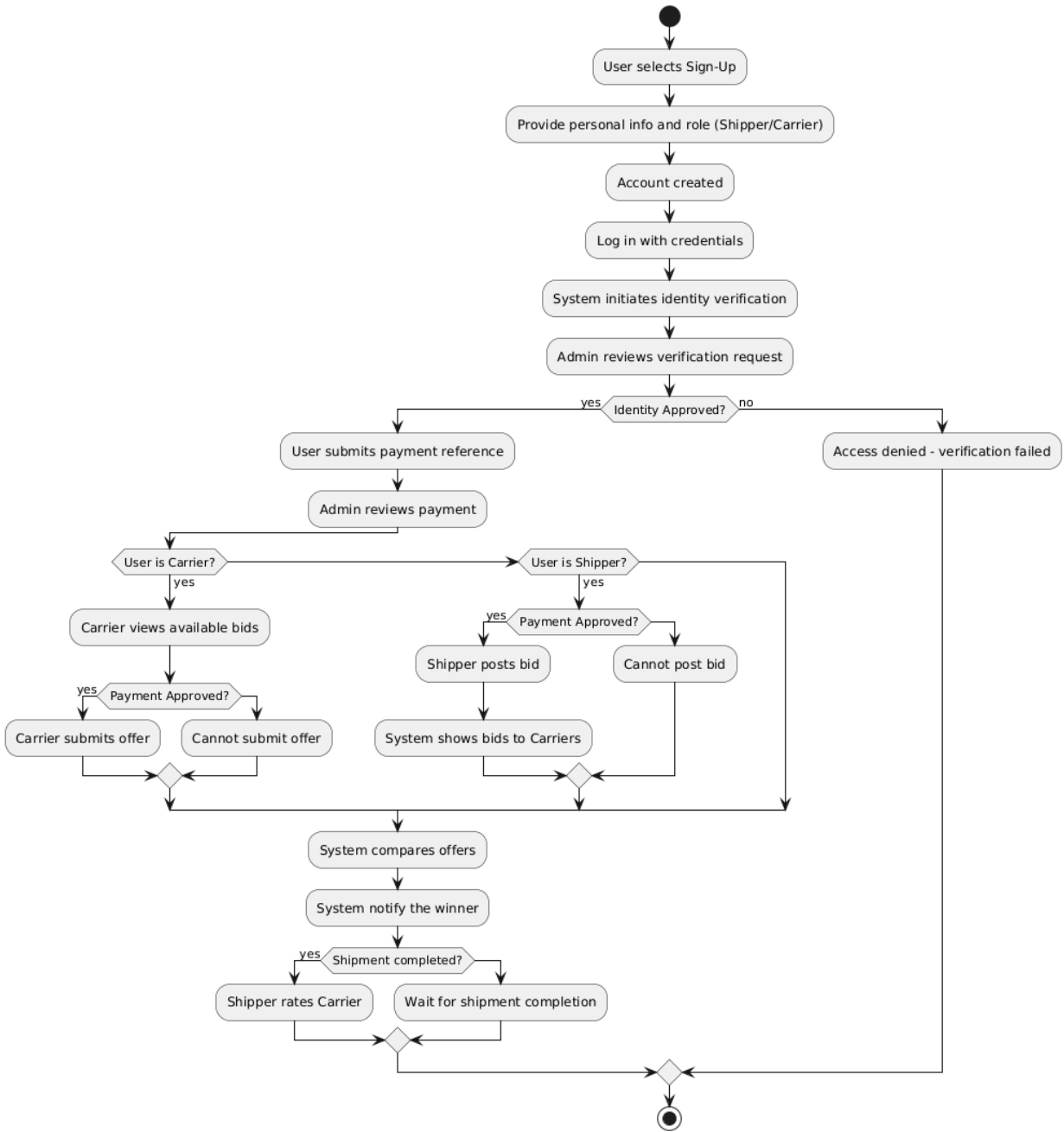


Fig of Active Diagram for All Actors 2.3.3

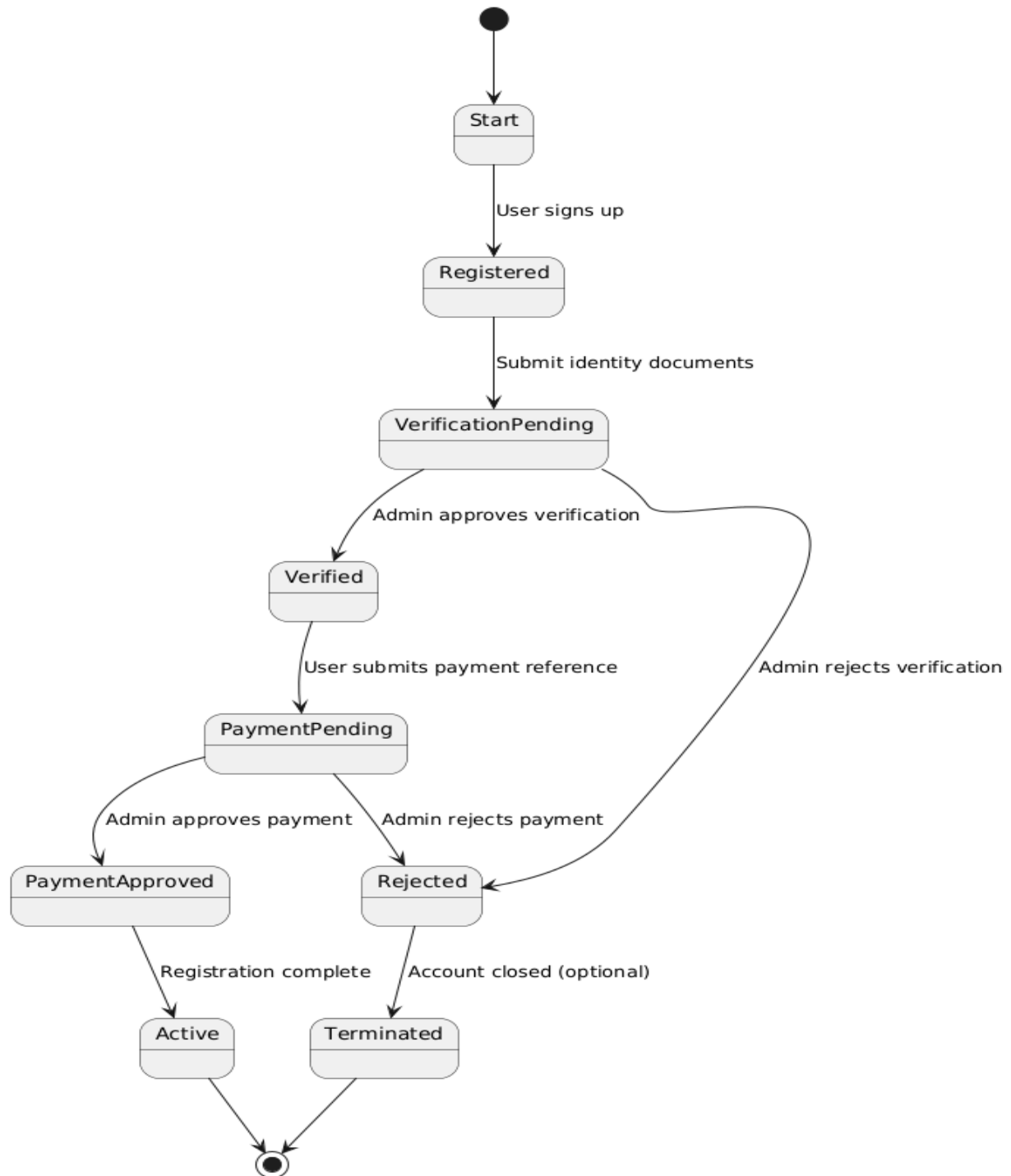


Fig of State chart for Registration 2.4

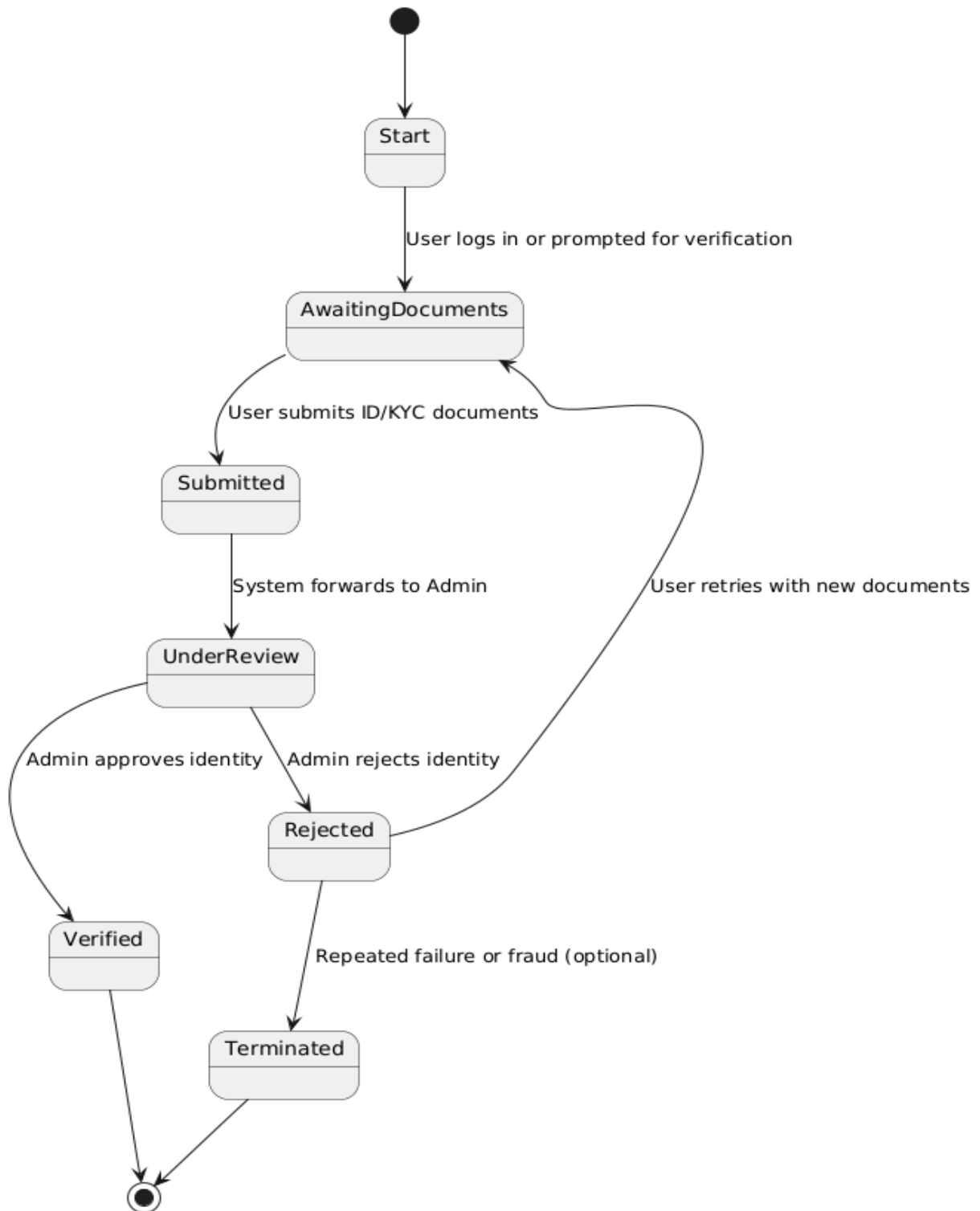


Fig of State chart for Identity Verification 2.4

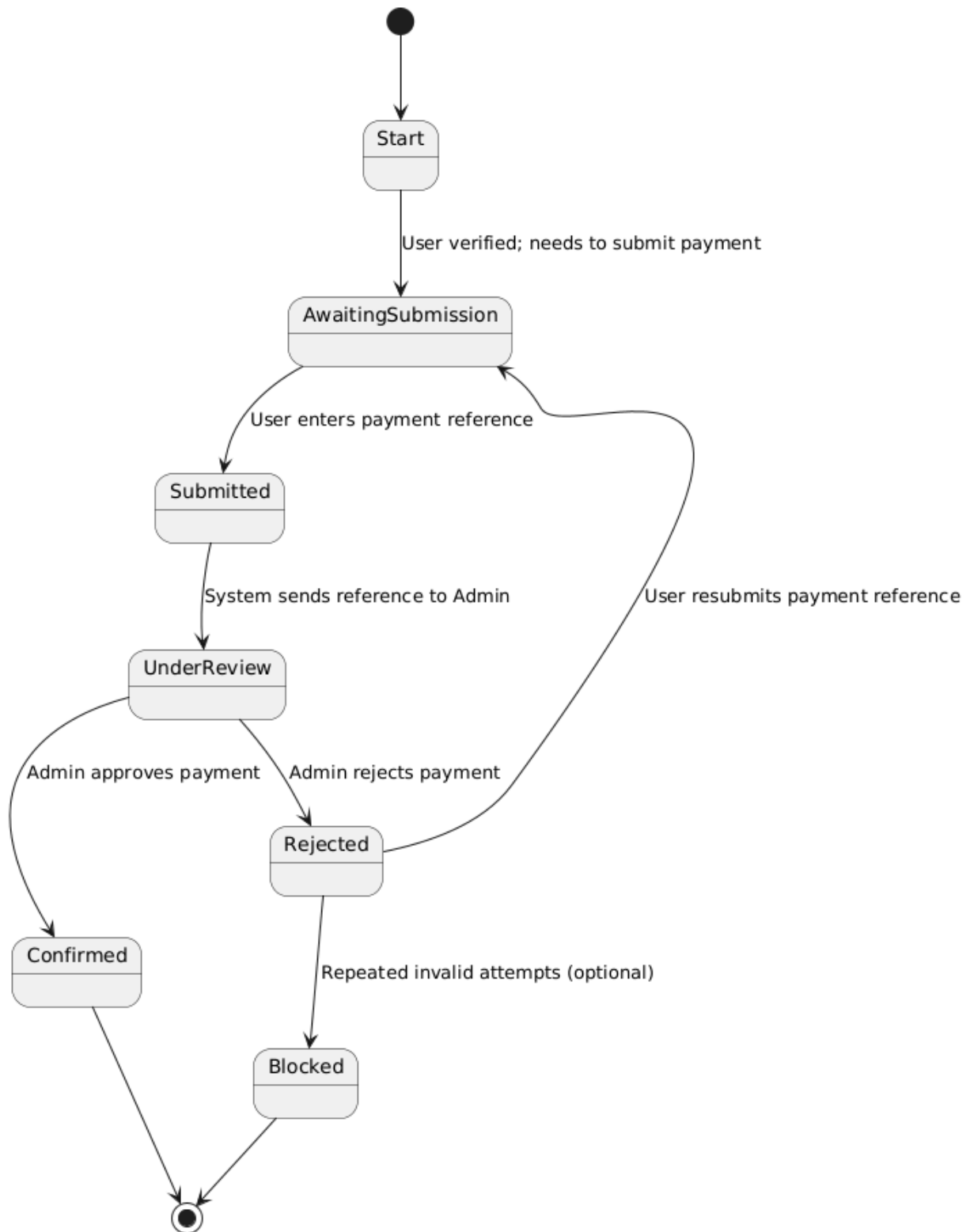
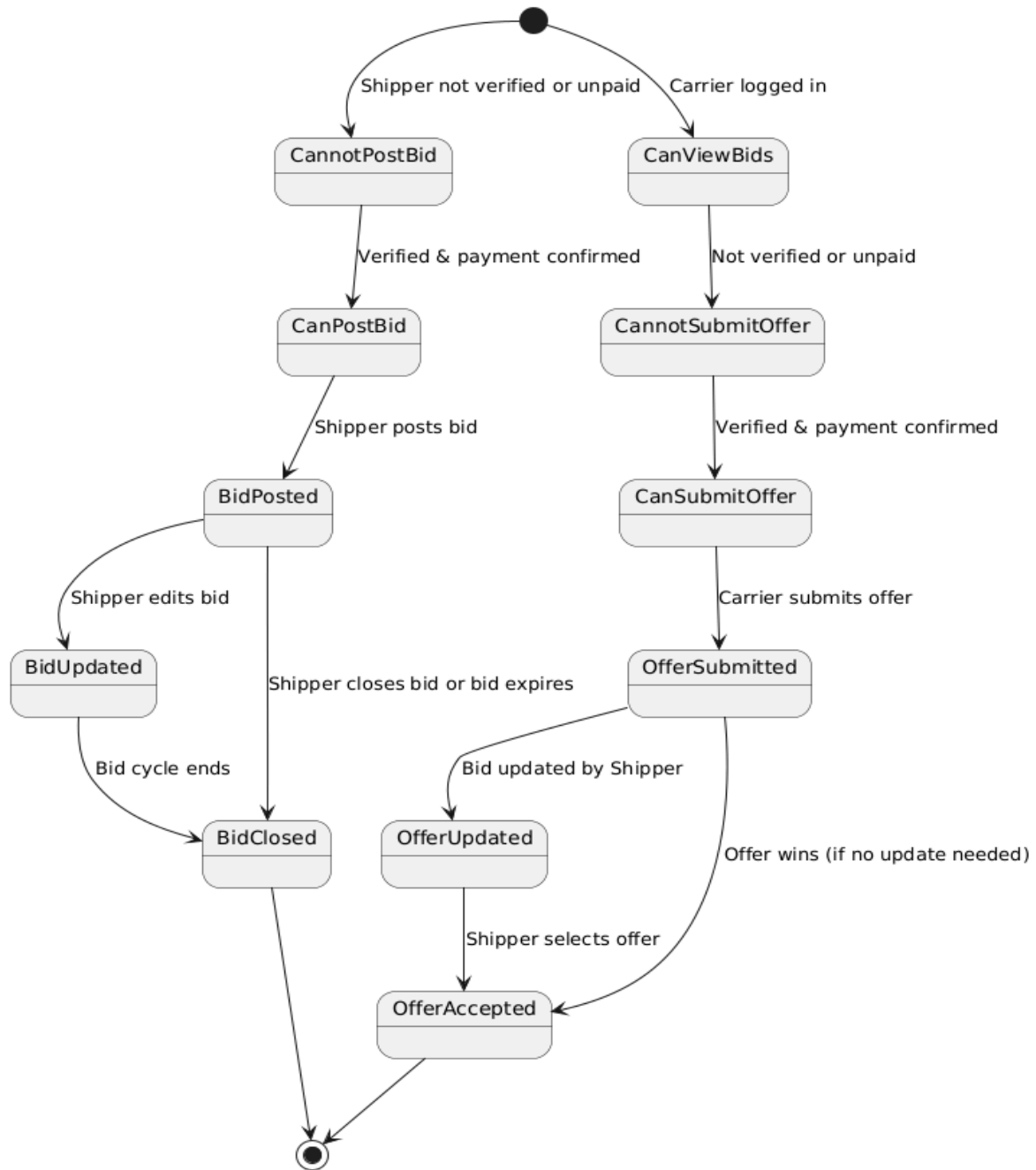
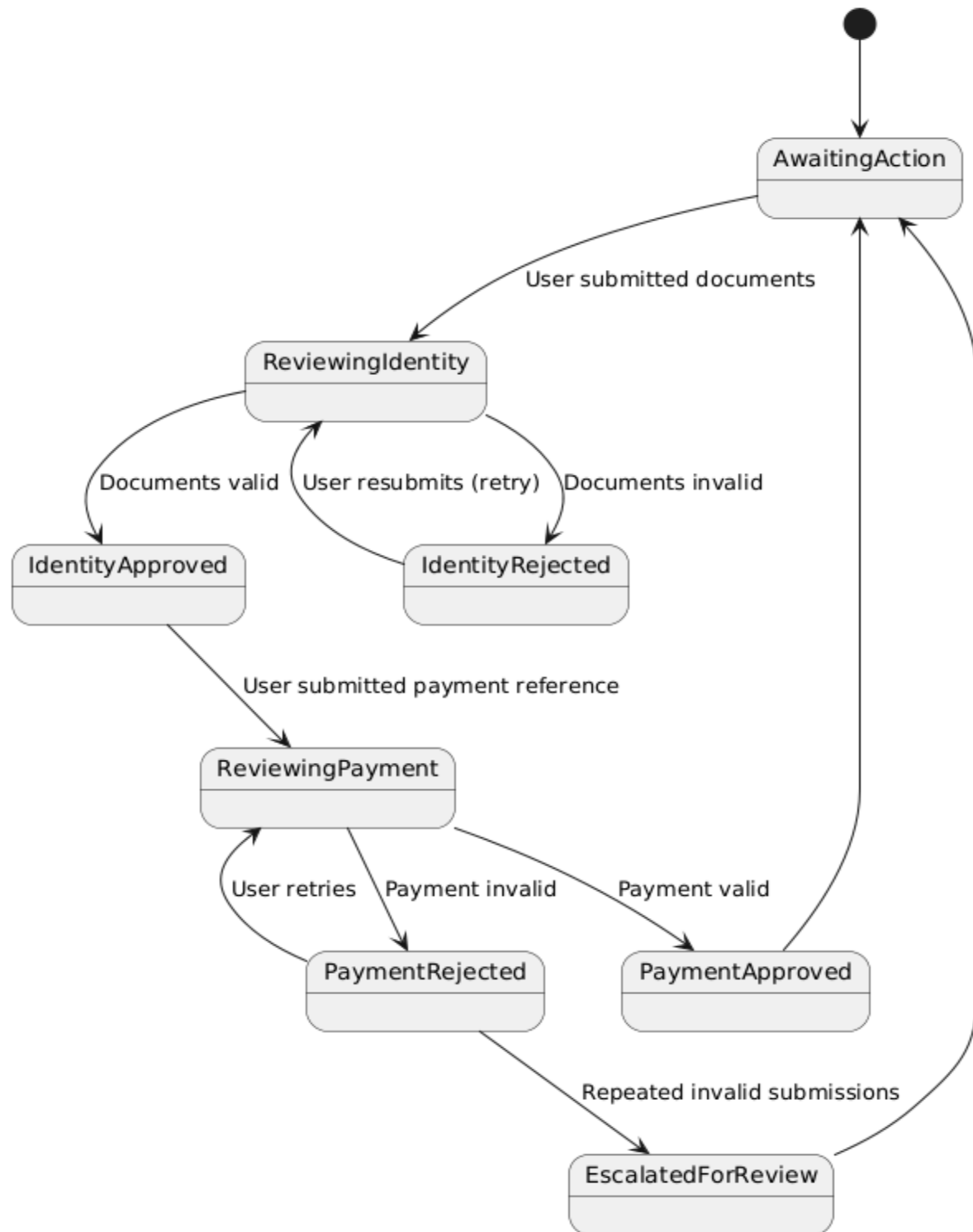


Fig of State chart for Payment Confirmation 2.4





Logistics and Cargo Bidding Platform - Class Diagram

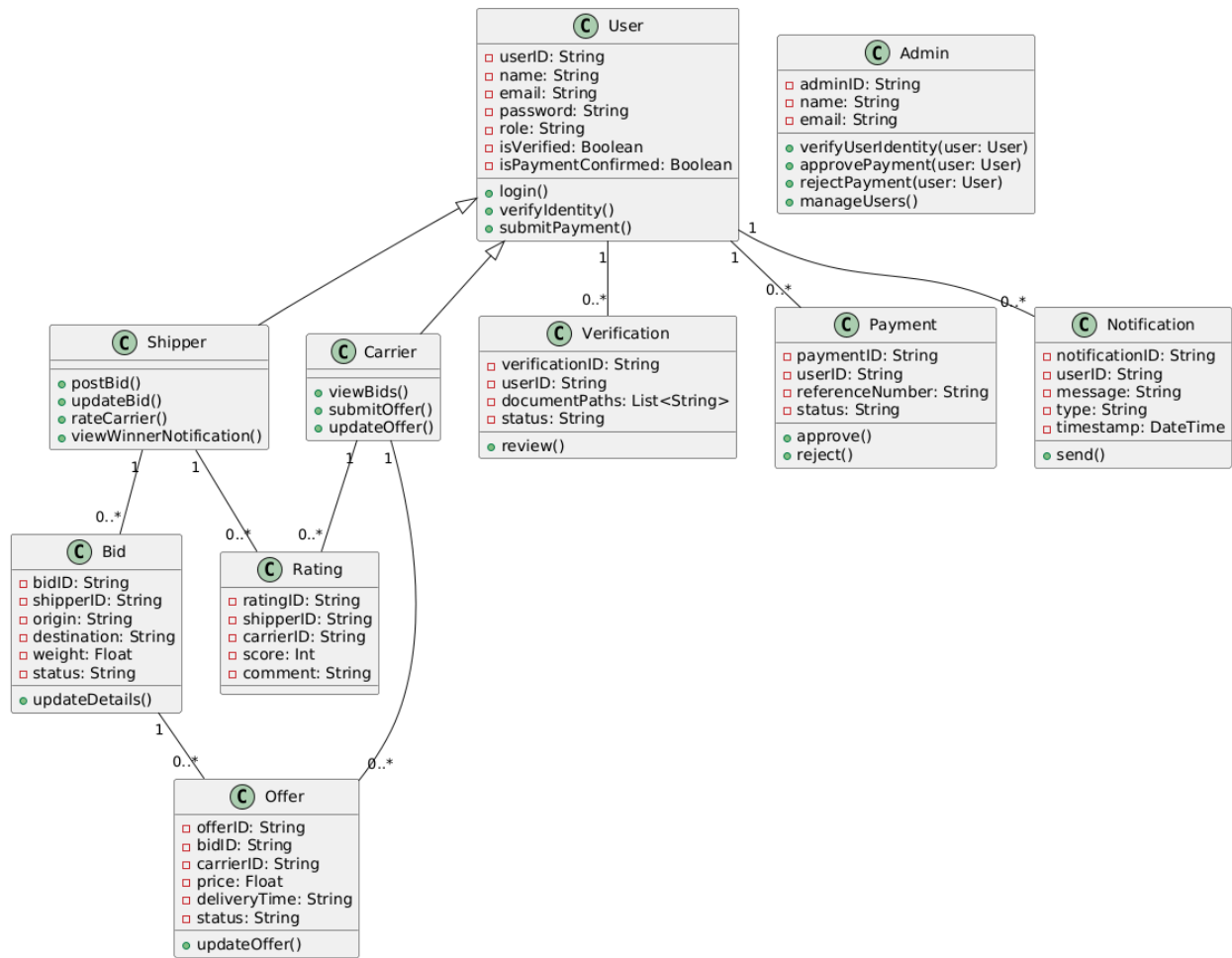


Fig of Class Diagram

2.4.3.6. Data Model

Entities

1. User

Field	Type	Description
user_id (PK)	UUID	Unique identifier
name	String	Full name
email	String	Unique email
password_hash	String	Hashed password
role	ENUM	Shipper or Carrier
is_verified	Boolean	Identity verification status
is_payment_confirmed	Boolean	Payment confirmation status
status	ENUM	active, suspended, etc.

2. Verification

Field	Type	Description
verification_id (PK)	UUID	Unique ID
user_id (FK)	UUID	References User
documents	Text/JSON	File paths or metadata

status	ENUM	pending, approved, rejected
submitted_at	Timestamp	When verification was submitted

3. Payment

Field	Type	Description
payment_id (PK)	UUID	Unique ID
user_id (FK)	UUID	References User
reference_number	String	Bank transfer reference
status	ENUM	pending, approved, rejected
submitted_at	Timestamp	Time of submission

4. Bid

Field	Type	Description
bid_id (PK)	UUID	Unique bid ID
shipper_id (FK)	UUID	References User (Shipper)
origin	String	Start location
destination	String	End location
weight	Float	Weight of the shipment

status	ENUM	open, closed, cancelled
created_at	Timestamp	Bid creation time

5. Offer

Field	Type	Description
offer_id (PK)	UUID	Unique offer ID
bid_id (FK)	UUID	References Bid
carrier_id (FK)	UUID	References User (Carrier)
price	Decimal	Offer price
delivery_time	String	Proposed delivery schedule
status	ENUM	pending, accepted, rejected
submitted_at	Timestamp	Offer submission time

6. Rating

Field	Type	Description
rating_id (PK)	UUID	Unique rating ID
shipper_id (FK)	UUID	Rater (Shipper)
carrier_id (FK)	UUID	Rated (Carrier)

score	Integer	1 to 5
comment	Text	Feedback
submitted_at	Timestamp	When rating was given

7. Notification

Field	Type	Description
notification_id (PK)	UUID	Unique notification ID
user_id (FK)	UUID	Receiver
message	Text	Message content
type	ENUM	system, payment, offer, etc.
timestamp	Timestamp	When notification was sent

Table of Data Model

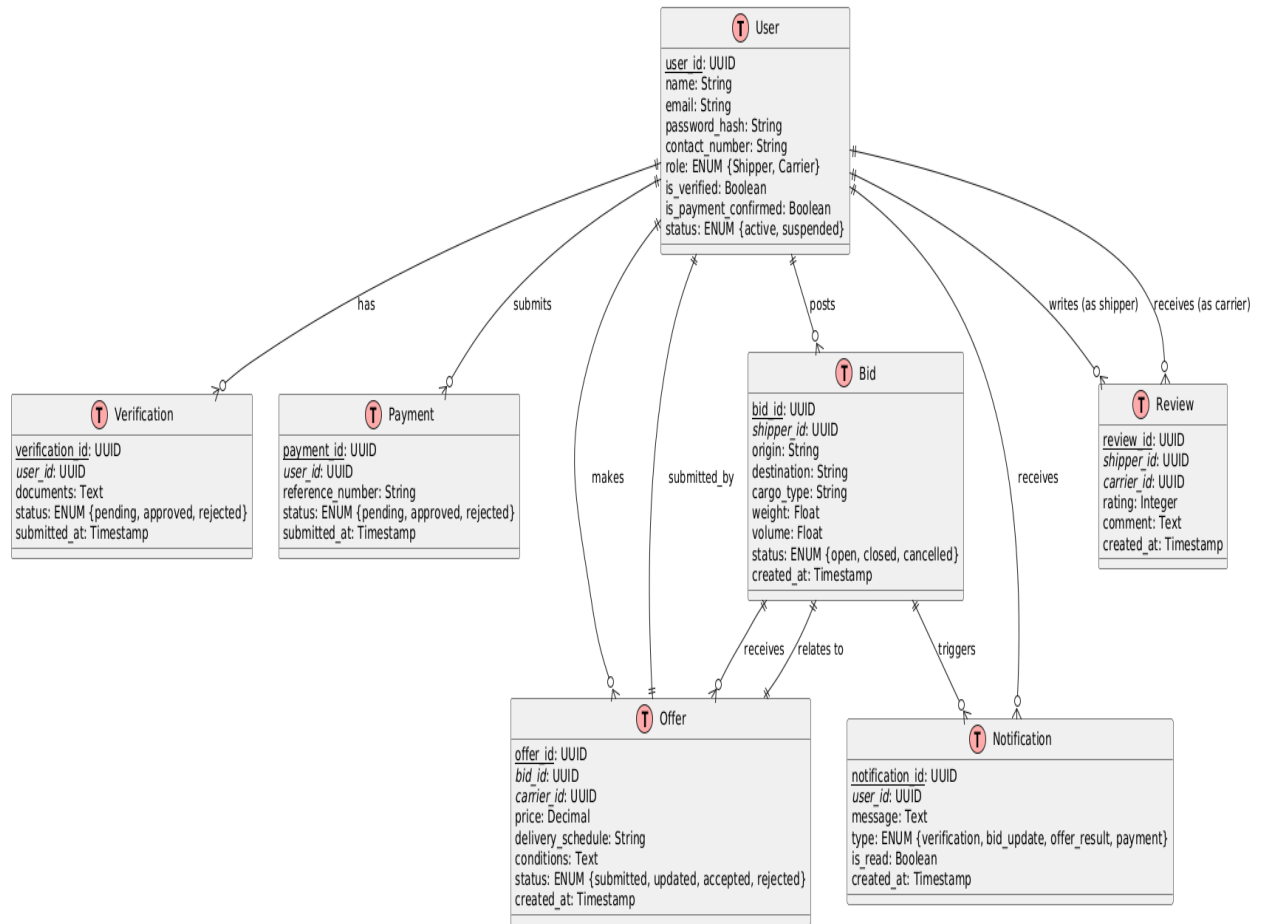


Figure for ER diagram

2.4.3.7. User Interface Prototyping

**Register**

Username:

Email

Phone Number:

Password:

Role:

Document upload: File
No file chosen

Next help? Visit our **help center**

**Login**

[Forgot Password?](#)

Don't have an account? **Register**

Next help? Visit our **help center**

<

Create Post

Cargo Type

Volume

Source

Destination

Date

Post

]

Chapter Three

3. System Design

3.1 Introduction

The Logistics and Cargo Bidding Platform is a web-based system designed to streamline the interaction between Shippers, Carriers, and Administrators in the transportation and logistics industry. Its core purpose is to simplify the cargo shipment process by allowing Shippers to post shipment bids and Carriers to submit competitive offers, all within a secure and transparent environment. The platform emphasizes user verification and payment confirmation as prerequisites for accessing core features, ensuring only trusted and authorized users can post bids or submit offers. Administrators play a crucial role in verifying user identities, approving payments, and managing system operations. The system is built on a modular and scalable architecture, supporting separate components for authentication, payment handling, bid and offer management, notifications, and administrative controls. It prioritizes security, usability, and maintainability, with a clear separation of concerns and role-based access. This design approach ensures that the platform can grow with user demand while maintaining reliability, performance, and trust.

3.2. Purpose of the system

The purpose of the proposed system is to provide a user-friendly, efficient, and secure platform for facilitating cargo shipment transactions between Shippers and Carriers. The system enables Shippers to post detailed bids for cargo transportation needs, while allowing Carriers to browse available bids and submit competitive offers. To ensure trust and operational integrity, the system enforces identity verification and payment confirmation before granting access to core features. Administrators are empowered to oversee user approvals, monitor platform activity, and manage data. Overall, the platform aims to streamline logistics coordination by automating manual tasks, reducing communication delays, and fostering a fair, transparent bidding environment.

3.3. Design Goals

The design goals of the Cargo and Logistics Transport Bidding System serve as the foundational principles that guide all technical and architectural decisions throughout the system's development. These goals ensure the system aligns with user needs, business objectives, and long-term scalability.

The design of the Logistics and Cargo Bidding Platform is guided by key objectives to ensure the system is **secure**, **scalable**, and **user-centric**. These goals help shape the architecture, component interaction, and user experience:

- **Maximize Usability:** Ensure that all user interfaces are intuitive and role-specific, allowing Shippers, Carriers, and Admins to perform their tasks efficiently with minimal training.
- **Ensure Data Security:** Implement strong authentication, identity verification, and secure data handling practices to protect sensitive user information and transaction details.
- **Optimize Performance:** Design the system to handle concurrent users and real-time operations such as bid postings, offer submissions, and notifications without delay or data loss.
- **Support Scalability:** Structure the system to accommodate growth in users, transactions, and services without compromising stability or speed.
- **Maintain Reliability:** Ensure continuous availability and fault tolerance so that users can depend on the system for mission-critical logistics operations.
- **Enable Maintainability:** Follow modular design and clean code practices to simplify updates, debugging, and future enhancements.

3.4 Current Software Architecture (if applicable)

At present, there is no centralized or integrated software platform supporting the logistics bidding process. Most interactions between Shippers and Carriers are conducted through manual methods, such as phone calls, emails, or spreadsheets. Information like bid details, offers, and payment confirmations are often recorded and shared inconsistently across different tools or platforms, leading to data fragmentation, lack of traceability, and operational inefficiency.

The current process lacks formal components or modules that support structured interactions. There is no shared user database, no identity or payment verification mechanism, and no automated notification system. This absence of an organized architecture makes it difficult to enforce access control, track transaction history, or ensure data accuracy and security.

The limitations of the current architecture include:

- **Poor scalability**, as it cannot support growing user demands or concurrent transactions.
- **Low reliability**, due to high dependence on manual input and human error.
- **Limited transparency**, which hinders trust between Shippers and Carriers.

- **Security risks**, from unverified users and untracked communications.

Given these challenges, a new software architecture is essential. It will automate workflows, ensure data integrity, enforce verification, and offer a seamless, role-based experience for all stakeholders, ultimately transforming the cargo bidding process into a more secure, efficient, and trustworthy digital platform.

3.5 Proposed Software Architecture

The proposed system is built on a modular, layered architecture that separates responsibilities into clearly defined components, enhancing scalability, security, and maintainability. It follows a three-tier architecture comprising:

- **Presentation Layer**: Interfaces for Shippers, Carriers, and Admins to interact with the system via web or mobile applications.
- **Application Layer**: Includes core services such as User Authentication, Identity Verification, Payment Handling, Bid and Offer Management, Notification Services, and Rating System. Each service is isolated to ensure independent scalability and ease of maintenance.
- **Data Layer**: A centralized relational database storing user profiles, bids, offers, payments, and audit logs with enforced constraints and security measures.

This architecture addresses key design goals such as usability (through role-based interfaces), security (via authentication, verification, and admin approval), performance (through service separation), and reliability (by minimizing single points of failure). It also supports future scalability, allowing the easy addition of new features or user roles.

In contrast to the current ad-hoc and manual process—which lacks structure, automation, and traceability—this architecture introduces automated workflows, real-time data consistency, and centralized monitoring, significantly improving operational efficiency and trust among users.

3.5.1 Subsystem Decomposition

To manage the complexity of the logistics and cargo bidding system, the architecture is divided into modular subsystems, each responsible for a distinct part of the overall functionality. This decomposition ensures maintainability, scalability, and separation of concerns. These subsystems will be represented as components in the component diagram.

Subsystems and Their Descriptions:

1. **User Authentication Service**

Handles user login, registration, and session management. Ensures only valid users can access the platform.

2. **Identity Verification Service**

Manages user verification through document submission or admin approval. Acts as a gatekeeper for access to critical features like posting bids or submitting offers.

3. **Payment Management Service**

Allows users to submit payment references and tracks payment status. Admins use this subsystem to verify payments before enabling core user actions.

4. **Bid Management Service (Shipper)**

Enables Shippers to post, update, or cancel shipment bids after verification and payment approval.

5. **Offer Management Service (Carrier)**

Allows Carriers to view bids and submit offers in response. Restricts offer updates unless the related bid is updated.

6. **Notification Service**

Sends alerts to users about verification, payments, winning bids, and offer updates. It supports real-time communication across the platform.

7. **Rating & Review System**

Allows Shippers to provide feedback and rate Carriers after successful shipments, improving service quality transparency.

8. **Admin Panel**

Central dashboard for admins to manage users, verify identity and payments, oversee system activity, and configure notifications.

9. **Bidding Comparison Engine**

Automatically compares offers submitted by Carriers and presents results to Shippers for final review.

10. **Database (Shared Data Layer)**

A centralized relational database stores user data, bids, offers, payments, ratings, and

system logs. Ensures data consistency and transactional integrity.

Dependencies Between Subsystems:

- The User Authentication Service is a prerequisite for all other services and must be successfully completed before accessing any core functionality.
- The Identity Verification Service is dependent on authentication and must approve a user before they can proceed to use the Bid or Offer subsystems.
- Payment Management must verify transactions before unlocking Bid Management and Offer Management for users.
- The Bidding Comparison Engine relies on data from both Bid Management and Offer Management to analyze and compare responses.
- The Notification Service interacts with nearly all subsystems to keep users informed in real time.
- The Admin Panel oversees and interacts with Verification, Payment, and Notification subsystems for approvals and system control.
- All subsystems read from and write to the shared Database, ensuring a unified data source for the platform.

3.5.2. Component diagram

Logistics and Cargo and Logistic Bidding Platform - Component Diagram

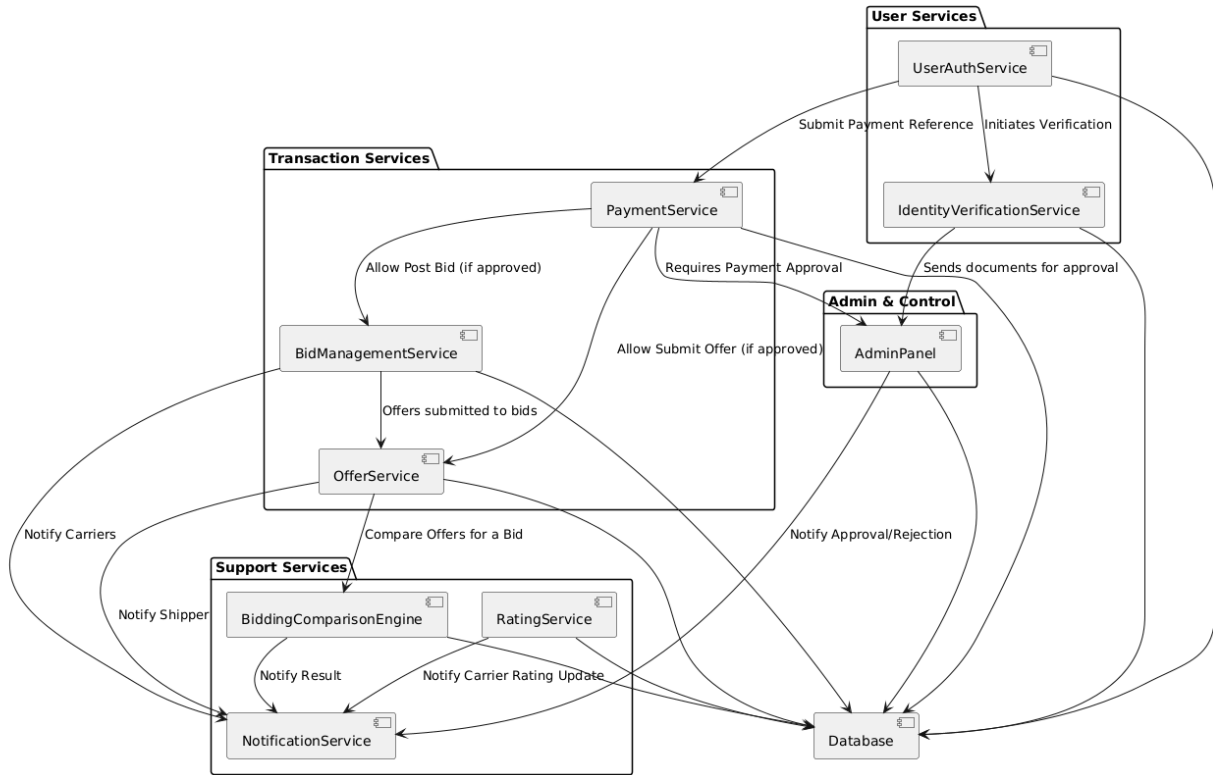


Fig of Component Diagram

3.5.3 Deployment Diagram

The deployment diagram illustrates the physical architecture of the logistics and cargo bidding platform, highlighting how software components are distributed across hardware nodes such as client devices, application servers, and database servers. The goal is to ensure efficient communication, scalability, and system reliability across all services.

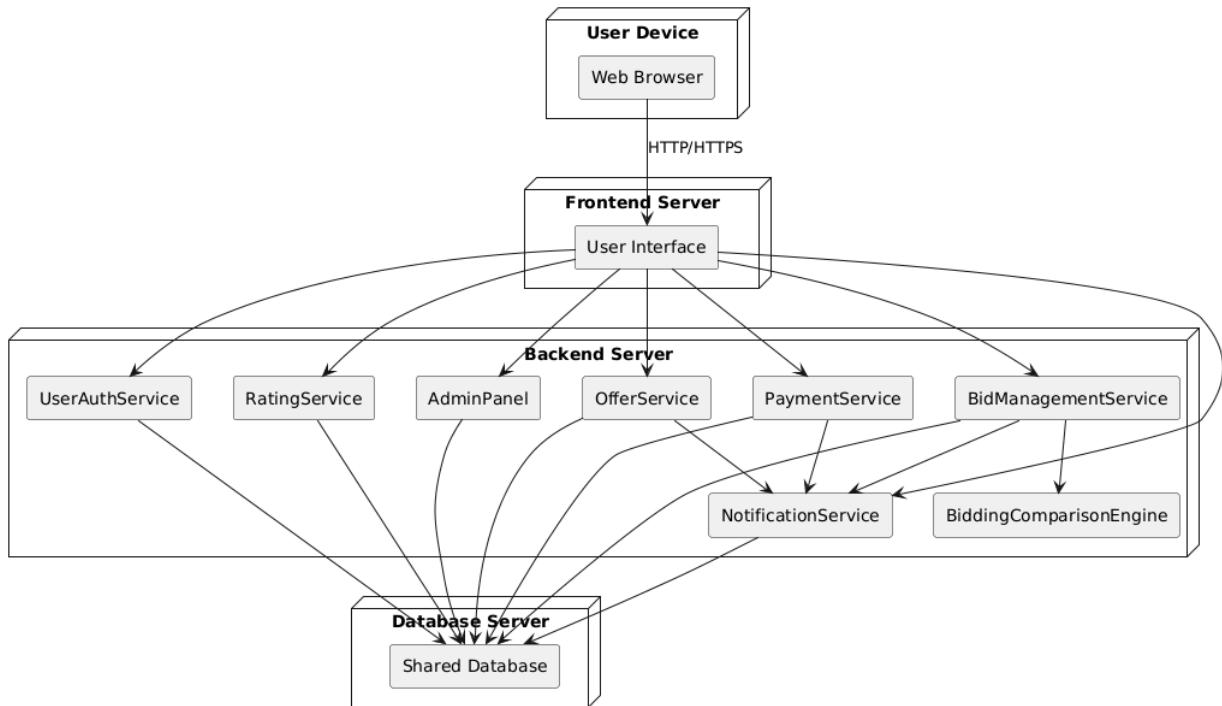


Fig of

Diagram

Deployment Overview

The system consists of the following physical nodes:

- **User Device (Client)** – Used by Shippers, Carriers, and Admins to interact with the system via a web interface.
- **Frontend Server** – Hosts the User Interface (UI) components.
- **Backend Server** – Handles the application logic for all services such as authentication, bidding, offers, payments, and notifications.
- **Database Server** – Hosts a centralized relational database that stores all user data, bids, offers, ratings, and payment records.

Deployment Strategy

- The **User Interface** is deployed on a dedicated frontend server to separate presentation logic from business logic.
- All core services (e.g., **UserAuthService**, **BidManagementService**, **OfferService**, **PaymentService**) are hosted on the **Backend Server**. These services are modular and can be deployed as containers or microservices for better maintainability and scaling.
- A single **Shared Database** resides on the **Database Server**, optimized for concurrent access, transactions, and backups.
- The system uses **HTTPS** communication between clients and servers to ensure data security.

Performance & Reliability Impact

- Separation of concerns between frontend, backend, and database layers improves scalability and fault isolation.
- Containerization of backend services (if applied) supports independent scaling and quick recovery in case of failure.
- Centralized database enables strong consistency but may become a bottleneck; future enhancements could include database replication or partitioning.
- The system can be deployed on cloud infrastructure (e.g., AWS, Azure) for high availability and global accessibility.

3.5.4 Persistent Data Management

The logistics and cargo bidding system manages various types of persistent data to ensure smooth operation, user accountability, and service integrity. This section outlines the types of data stored, storage technologies used, access strategies, and the approach to data integrity and recovery.

Types of Persistently Stored Data

The system will persistently store the following data types:

- **User Information:** Names, email addresses, roles (Shipper/Carrier/Admin), login credentials (hashed), contact details.

- **Verification Documents:** Government-issued IDs, licenses, or proof of business registration (stored securely, possibly as encrypted file blobs).
- **Bids and Offers:** Details of shipment requests (origin, destination, weight), offers submitted by carriers (price, schedule).
- **Payment Records:** Payment references, statuses (pending, approved), admin verification logs.
- **Ratings and Reviews:** Feedback provided by shippers after shipment completion.
- **System Notifications:** Logs of system-generated notifications to users (e.g., bid updates, verification approvals).
- **Audit Logs:** For security, user activity history, and compliance purposes.

Database Technology & Storage Mechanism

- **Relational Database Management System (RDBMS):** A robust RDBMS such as PostgreSQL or MySQL is used for structured data due to the transactional nature and relational dependencies between entities (users, bids, offers).

Data Access and Management

- The system uses an ORM (Object-Relational Mapping) layer like TypeORM, Hibernate, or Sequelize for secure and efficient database access.
- Services (e.g., UserAuthService, PaymentService) access data through defined repositories or service layers to ensure encapsulation and access control.
- Role-based access controls (RBAC) prevent unauthorized access or modification of data.

Data Backup and Recovery

- Automated daily backups of the database are scheduled and stored securely in a separate location.
- For mission-critical data (payment records, verification documents), incremental backups are also implemented.

- A backup retention policy ensures that data is retained for a configurable period (e.g., 30–90 days).
- Disaster Recovery (DR) plans include full-database restoration scripts and procedures tested periodically.

Data Integrity and Consistency

- Database **constraints**, **foreign keys**, and **transactions** are used to maintain data consistency across entities.
- Sensitive data (like passwords) is **hashed using bcrypt or Argon2**.
- Administrative actions (e.g., verification approvals) are **logged with timestamps** to maintain audit trails.
- Application-level validation ensures no malformed or incomplete data is stored.

3.5.5 Detailed Database Design

3.5.5.1 Relational Tables

The following relational tables represent the detailed schema of the system’s database. Each table includes its primary key, foreign keys, and data types to ensure referential integrity and support the platform’s operations.

```
from django.contrib import admin
```

```
from django.conf import settings
```

```
from django.db import models
```

```
from django.utils import timezone
```

```
# cargobid/model.py
```

```
class Customer(models.Model):
```

```
    ROLE_CHOICES = [
```

```

        ('shipper', 'Shipper'),

        ('carrier', 'Carrier'),

    ]

    user = models.OneToOneField(settings.AUTH_USER_MODEL,
on_delete=models.CASCADE)

    phone = models.CharField(max_length=15)

    role = models.CharField(max_length=20, choices=ROLE_CHOICES)


    def str(self):

        return f"{self.user.first_name} {self.user.last_name}"


    @admin.display(ordering='user__first_name')

    def first_name(self):

        return self.user.first_name


    @admin.display(ordering='user__last_name')

    def last_name(self):

        return self.user.last_name

    class Meta:

        ordering = ['user__first_name', 'user__last_name']


class Address(models.Model):

    customer = models.ForeignKey(Customer, on_delete=models.CASCADE,
related_name='addresses')
```

```

street = models.CharField(max_length=100)

city = models.CharField(max_length=100)

zip_code = models.CharField(max_length=20, null=True, blank=True)

country = models.CharField(max_length=100)


class CarrierProfile(models.Model):

    CARRIER_TYPE_CHOICES = [

        ('company', 'Company'),

        ('plc', 'PLC'),

        ('truck_owner', 'Truck Owner'),

    ]

    customer = models.OneToOneField(Customer, on_delete=models.CASCADE,
related_name='carrier_profile', limit_choices_to={'role': 'carrier'})

    carrier_type = models.CharField(max_length=20, choices=CARRIER_TYPE_CHOICES)

    company_name = models.CharField(max_length=100)

    registration_date = models.DateTimeField(default=timezone.now)

    is_profile_approved = models.BooleanField(default=False)

    is_documents_verified = models.BooleanField(default=False)

    is_payment_verified = models.BooleanField(default=False)

    def str(self):

        return f"{self.company_name} ({self.get_carrier_type_display()})"

    # Documents only for 'company'

```

```

class ShareCompanyProfile(models.Model):

    carrier_profile = models.OneToOneField(CarrierProfile, on_delete=models.CASCADE,
related_name='share_company_profile')

    company_business_registration_doc =
models.FileField(upload_to='documents/share_company/')

    company_business_license_doc = models.FileField(upload_to='documents/share_company/')

    company_competency_certificate_doc =
models.FileField(upload_to='documents/share_company/')

    company_tax_clearance_doc = models.FileField(upload_to='documents/share_company/')

    company_vat_certificate_doc = models.FileField(upload_to='documents/share_company/')

    company_number_of_trucks = models.PositiveIntegerField()

```

Documents only for 'Truck Owner'

```

class TruckOwnerProfile(models.Model):

    carrier_profile = models.OneToOneField(CarrierProfile, on_delete=models.CASCADE,
related_name='truck_owner_profile')

    truck_libreh_number = models.CharField(max_length=50)

    truck_business_license_doc = models.FileField(upload_to='documents/truck_owner/')

    truck_tin_number = models.CharField(max_length=50)

```

Documents only for 'plc'

```

class PLCProfile(models.Model):

    carrier_profile = models.OneToOneField(CarrierProfile, on_delete=models.CASCADE,
related_name='plc_profile')

    plc_registration_doc = models.FileField(upload_to='documents/share_plc/')

```

```

plc_business_license_doc = models.FileField(upload_to='documents/share_plc/')

plc_competency_certificate_doc = models.FileField(upload_to='documents/share_plc/')

plc_tax_clearance_doc = models.FileField(upload_to='documents/share_plc/')

plc_vat_certificate_doc = models.FileField(upload_to='documents/share_plc/')

plc_number_of_trucks = models.PositiveIntegerField()

class ShipperProfile(models.Model):

    customer = models.OneToOneField(Customer, on_delete=models.CASCADE,
related_name='shipper_profile', limit_choices_to={'role': 'shipper'})

    business_license_doc = models.FileField(max_length=50)

    tin_number = models.CharField(max_length=50)

    company_name = models.CharField(max_length=100)

    registration_date = models.DateTimeField(default=timezone.now)

    is_profile_approved = models.BooleanField(default=False)

    is_documents_verified = models.BooleanField(default=False)

    is_payment_verified = models.BooleanField(default=False)


class Payment(models.Model):

    ROLE_CHOICES = [

        ('shipper', 'Shipper'),

        ('carrier', 'Carrier'),

    ]

    customer = models.ForeignKey(Customer, on_delete=models.CASCADE,
related_name="payments")

    role = models.CharField(max_length=20, choices=ROLE_CHOICES)

```



```

payment_code = models.CharField(max_length=100) # serial number from bank or receipt

amount = models.DecimalField(max_digits=12, decimal_places=2)

created_at = models.DateTimeField(auto_now_add=True)

approved = models.BooleanField(default=False)

approved_at = models.DateTimeField(null=True, blank=True)


class Bid(models.Model):

    shipper = models.ForeignKey(ShipperProfile, on_delete=models.CASCADE,
related_name='bids')

    title = models.CharField(max_length=255)

    description = models.TextField()

    origin = models.CharField(max_length=200)

    destination = models.CharField(max_length=200)

    cargo_type = models.CharField(max_length=100)

    weight_kg = models.FloatField()

    created_at = models.DateTimeField(auto_now_add=True)

    cpo_amount = models.DecimalField(max_digits=10, decimal_places=2)

    deadline = models.DateTimeField(null=True, blank=True)

    # optional status field

    status = models.CharField(max_length=20, choices=[('open', 'Open'), ('assigned', 'Assigned'),
('closed', 'Closed')], default='open')

    def str(self):

        return f'{self.title} - {self.shipper.company_name}'

```

```

class Offer(models.Model):

    bid = models.ForeignKey(Bid, on_delete=models.CASCADE, related_name="offers")

    carrier = models.ForeignKey(CarrierProfile, on_delete=models.CASCADE,
related_name="offers")

    # Bid amount

    amount = models.DecimalField(max_digits=12, decimal_places=2)

    # Bank details Cpo

    bank_name = models.CharField(max_length=100)

    bank_account_number = models.CharField(max_length=50)

    created_at = models.DateTimeField(auto_now_add=True)

    approved = models.BooleanField(default=False)

```

```

class ReviewAndRate(models.Model):

    carrier = models.ForeignKey(CarrierProfile, on_delete=models.CASCADE,
related_name='carrier_shippers')

    name = models.CharField(max_length=255)

    rating = models.PositiveSmallIntegerField(blank=True, null=True) # 1-5 scale

    description = models.TextField()

    date = models.DateTimeField(auto_now_add=True)

```

--	--	--	--

1. Users

Column Name	Data Type	Key	Description
user_id	INT	Primary Key	
name	VARC HAR(100)		Full name of the user
email	VARC HAR(150)	Unique	User's email address
password_hash	VARC HAR(255)		Hashed password

role	ENUM	'Shipper', 'Carrier', 'Admin'
phone_number	VARCHAR(20)	User's contact number
is_verified	BOOLEAN	Identity verification status
is_active	BOOLEAN	Account status
created_at	TIMESTAMP	Account creation timestamp

2. VerificationDocuments

Column Name	Data Type	Key	Description
document_id	INT	Primary Key	Unique ID for the document
user_id	INT	Foreign Key	References <code>Users(user_id)</code>

document_type	VARCHAR(100)	Type of document (e.g., ID, License)
file_path	VARCHAR(255)	Path to stored document file
status	ENUM	'Pending', 'Approved', 'Rejected'
upload_date	TIMESTAMP	Upload date and time

3. Payments

Column Name	Data Type	Key	Description
payment_id	INT	Primary Key	Unique identifier for payment
user_id	INT	Foreign Key	References <code>Users(user_id)</code>
reference_number	VARCHAR(100)		Bank-provided reference number

status	ENUM		'Pending', 'Approved', 'Rejected'
submitted_at	TIMESTAMP		Time of submission
approved_by	INT	Foreign Key	Admin who verified (Users.user_id)

4. Bids

Column Name	Data Type	Key	Description
bid_id	INT	Primary Key	Unique ID for the bid
shipper_id	INT	Foreign Key	References <code>Users(user_id)</code>
origin	VARCHAR(255)		Starting location
destination	VARCHAR(255)		Ending location

weight_ kg	DECIMA L(10,2)	Cargo weight in kilograms
descript ion	TEXT	Optional description
status	ENUM	'Open', 'Closed', 'Canceled'
created _at	TIMEST AMP	Time of bid creation

5. Offers

Column Name	Data Type	Key	Description
offer_id	INT	Prim ary Key	Unique offer ID
carrier_ id	INT	Forei gn Key	References <code>Users(user_id)</code>
bid_id	INT	Forei gn Key	References <code>Bids(bid_id)</code>

offer_price	DECIMAL(10,2)	Offered price by the carrier
delivery_time	VARCHAR(100)	Estimated delivery schedule
special_terms	TEXT	Any special conditions
is_selected	BOOLEAN	Marks if this offer was accepted
submitted_at	TIMESTAMP	Time of offer submission

6. Ratings

Column Name	Data Type	Key	Description
rating_id	INT	Primary Key	Unique identifier
shipper_id	INT	Foreign Key	References <code>Users(user_id)</code>

carrier_id	INT	Foreign Key	References <code>Users(user_id)</code>
bid_id	INT	Foreign Key	References <code>Bids(bid_id)</code>
rating_score	INT		Rating from 1 to 5
review_comment	TEXT		Optional written review
created_at	TIMESTAMP		Time of review submission

7. Notifications

Column Name	Data Type	Key	Description
notification_id	INT	Primary Key	Unique notification ID

user_id	INT	Foreign Key	References Users(user_id)
message	TEXT		Notification message
is_read	BOOLEAN		Mark if the user read it
created_at	TIMESTAMP		Notification creation time

Table of Relation

3.5.5.3 Extended Entity-Relationship Diagram (EER)

Key EER Concepts Used:

- **Specialization/Generalization:** User is a general entity; Shipper and Carrier are specialized subtypes.
- **Relationships:** Bids, Offers, Payments, Ratings, and Admin Approvals link key entities.
- **Participation constraints:** Some are total (e.g., every Bid must belong to a Shipper).

Conceptual Design

Entities:

- User → Specialized into Shipper and Carrier

- Notification

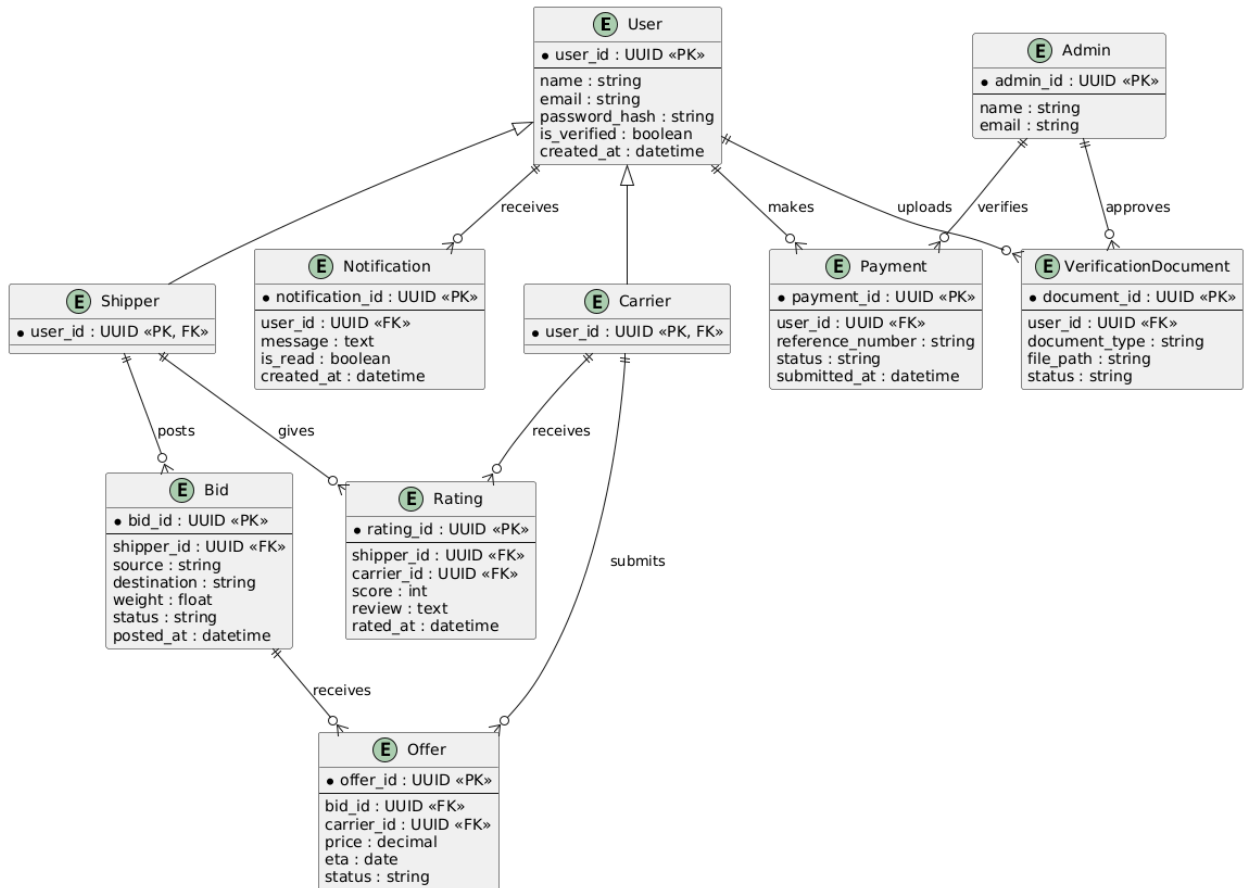


Fig of EER

3.5.5.4 OO-Relational Mapping

Object-Relational Mapping (ORM) is the process of linking classes (in Object-Oriented Programming) to database tables (in a Relational Database). This section demonstrates how the object model (your system's classes) maps onto your relational schema.

Example Class-to-Table Mapping

Mapped Table		Attributes (Properties → Columns)	Relationships
User	users	user_id, name, email, password, role, is_verified	1-to-1 with Verification, 1-to-many with Payments, Bids
Shipper	Inherits User		Specialization of User
Carrier	Inherits User		Specialization of User
Bid	bids	bid_id, shipper_id, source, destination, weight, created_at	Many-to-1 with Shipper, 1-to-many with Offers
Offer	offers	offer_id, carrier_id, bid_id, price, delivery_schedule, status	Many-to-1 with Bid and Carrier

Payment	payments	payment_id, user_id, reference_number, status, confirmed_by_admin	Many-to-1 with User
Verification	verifications	verification_id, user_id, method, status, verified_at	1-to-1 with User
Admin	admin	admin_id, name, email, password	
Rating	ratings	rating_id, shipper_id, carrier_id, score, review, created_at	Many-to-1 with Shipper and Carrier
Notification	notifications	notification_id, user_id, message, type, created_at, is_read	Many-to-1 with User

Table of OO-Relational Mapping

Mapping Rules Applied

- **Inheritance:** `Shipper` and `Carrier` are subclasses of `User`. In the relational model, this can be handled via:
 - Single Table Inheritance: Store both in one `users` table with a `role` column.
- **Associations:**
 - `User` ↔ `Bid`: One-to-many (a shipper can post many bids).
 - `Bid` ↔ `Offer`: One-to-many (a bid can have many offers).

- User ↔ Payment: One-to-many (a user can have multiple payment records).
- Admin verifies Payments and Users: Captured via `confirmed_by_admin` fields or a join table for audit trails.

3.5.6 Access Control and Security

To protect sensitive user data and ensure that only authorized users can access or manipulate system resources, the proposed system integrates robust access control and security mechanisms.

User Authentication

The system employs secure user authentication methods to validate the identity of each user:

- Login credentials (username and password) are required for all users.
- Passwords are securely stored using hashing algorithms (e.g., bcrypt) to prevent unauthorized access even in case of a data breach.
- Multi-Factor Authentication (MFA) may be incorporated to provide an additional layer of protection, especially for administrative users.

User Authorization

A Role-Based Access Control (RBAC) model governs user permissions:

- **Shippers** are authorized to post and update bids, view carrier offers, and rate carriers.
- **Carriers** are authorized to view bids, submit and update offers, and receive feedback.
- **Administrators** have full access to system functionalities including managing users, approving verifications and payments, and monitoring platform activity.
Authorization is enforced at both the application and database levels to prevent privilege escalation.

Data Privacy and Encryption

To safeguard sensitive information, the system applies comprehensive data privacy and encryption measures:

- All data transmission between client and server is secured using HTTPS with TLS encryption.

- Sensitive data such as identity documents, payment information, and login credentials are encrypted at rest using AES-256 encryption.
- File access is restricted based on user roles and verified status to minimize exposure of private data.

System Monitoring and Audit Trails

- The system logs key activities including user logins, failed access attempts, data updates, and administrative actions.
- These logs support both security audits and real-time monitoring of suspicious behavior.

Backup and Recovery

- The database is backed up regularly to ensure data integrity and availability.
- In the event of a system failure, data recovery procedures are in place to restore the system to the most recent stable state with minimal downtime.

Access Control Enforcement

- Access control is enforced through secure backend APIs and front-end route protection.
- Unauthorized access attempts are blocked and logged, with potential alerts triggered for admin review.

3.5.7 Global Software Control

The global software control architecture defines how the system manages the overall control flow, ensures modular coordination, and handles request synchronization across various subsystems.

Control Flow Overview

The system adopts a client-server model with a layered architecture, where:

- The User Interface (UI) layer initiates user actions (e.g., login, post bid, submit offer).
- The Application Logic Layer processes requests and enforces business rules.

- The Service Layer (e.g., UserAuthService, PaymentService, BidManagementService) executes core functionalities.
- The Data Access Layer communicates with the database.

Request Handling

- Requests originate from users (Shippers, Carriers, Admins) via the UI and are sent to the backend services.
- A central Requestdispatcher routes requests to appropriate services (e.g., a login request is routed to the UserAuthService; bid posting to BidManagementService).
- Services validate input, check preconditions (e.g., identity verified, payment confirmed), and trigger necessary business logic.
- After processing, responses (success/failure messages, data) are sent back to the UI for user feedback.

Synchronization Between Subsystems

- Asynchronous communication is used for non-blocking operations like sending notifications or logging activities (via NotificationService and AdminPanel).
- Synchronous communication is enforced for time-sensitive operations (e.g., verifying login credentials, confirming payment).
- Shared state is maintained via a centralized database, ensuring consistency of user, bid, and offer data across services.

Control Components

- **Authentication Gateway:** Controls access to all services based on login and role.
- **Verification Middleware:** Ensures only verified users proceed to sensitive operations like bidding or submitting offers.
- **Transaction Coordinator:** Handles multi-step workflows like posting a bid and notifying carriers.

This global control structure ensures that:

- **User interactions are efficiently handled.**
- **Subsystems remain loosely coupled but tightly coordinated.**
- **Scalability, maintainability, and fault tolerance** are upheld throughout the software lifecycle.

3.5.8 Boundary Conditions

This section outlines how the system will behave under boundary conditions such as startup, shutdown, error scenarios, and extreme loads. It also details how the system is designed to handle these cases gracefully to ensure stability, availability, and reliability.

System Startup

- On initialization, the system will load essential configuration files, establish database connections, and activate core services such as UserAuthService, NotificationService, and AdminPanel.
- Health checks will be performed on all critical components before the system transitions to the “Ready” state.
- If any critical service fails to initialize, the system will enter a maintenance mode and notify the system administrator.

System Shutdown

- During shutdown, the system will first disable new user requests.
- All active sessions will be closed gracefully after completing in-progress operations (e.g., offer submission, payment verification).
- Services will flush pending logs, save the latest system state, and disconnect cleanly from the database.
- The system ensures data consistency and no transactional loss during the shutdown.

Error Handling

- Authentication errors (e.g., invalid login, unauthorized access) will return appropriate HTTP status codes (401/403) and display friendly error messages to users.

- Payment errors (e.g., invalid reference, timeout) will be logged and sent to the AdminPanel for manual resolution.
- Service failures are isolated using circuit breakers to prevent cascading crashes.
- All errors are logged using a centralized logging mechanism, and critical errors trigger alerts to the admin via the NotificationService.

Extreme Load Conditions

- In cases of high user traffic (e.g., peak hours or during high-demand shipment periods):
 - Load balancers will distribute requests across multiple instances of services (horizontal scaling).
 - Caching mechanisms will be used for read-heavy endpoints like viewing posted bids.
 - Rate limiting and throttling will be applied to prevent system abuse and ensure fair usage.
 - A graceful degradation strategy will reduce non-critical functionality (e.g., real-time notifications) to prioritize core services.

Unusual Scenarios

- Mass payment submissions: Queuing and batching will be applied to admin validation operations to maintain performance.
- Simultaneous bid/offer updates: Concurrency control (e.g., optimistic locking) will be enforced to prevent race conditions.
- Database unavailability: The system will enter a read-only failover mode, informing users of degraded functionality while ensuring no data loss.

Resilience and Recovery

- The system implements automatic recovery strategies such as retrying failed requests and restarting failed services.

- Data backups are scheduled regularly, and a disaster recovery plan ensures system restoration in case of critical failure.
- For front-end users, clear and non-technical error messages will guide them through corrective actions or alternative paths.